1. Setup

```
# Install packages that are not always preinstalled in a fresh Colab runtime.
%pip install -q lightgbm xgboost shap optuna kaleido holidays duckdb
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 442.4/442.4 kB 8.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 55.8/55.8 kB 5.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 268.7/268.7 kB 26.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 58.0/58.0 kB 5.9 MB/s eta 0:00:00
```

```
import os
import gc
import json
import time
import warnings
import zipfile
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
import joblib
import duckdb
import holidays
import shap
import optuna

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    roc_auc_score, f1_score, precision_score, recall_score, brier_score_loss,
    roc_curve, confusion_matrix, mean_absolute_error, mean_squared_error, r2_score,
)
from sklearn.calibration import calibration_curve
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier, LGBMRegressor

warnings.filterwarnings("ignore")
optuna.logging.set_verbosity(optuna.logging.WARNING)

sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams["figure.dpi"] = 110
plt.rcParams["figure.facecolor"] = "white"

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print("Setup complete.")
```

```
Setup complete.
```

```
RF_N_ESTIMATORS = 200          # Random Forest trees (both baseline + per-fold)
XGB_LGBM_N_ESTIMATORS = 300    # boosting rounds for un-tuned baseline runs
OPTUNA_TRIALS_CLASSIFIER = 25  # HPO trials per classifier (XGBoost, LightGBM)
OPTUNA_TRIALS_REGRESSOR = 20   # HPO trials for the LightGBM duration regressor
SHAP_SAMPLE_SIZE = 8000        # rows sampled from the test set for SHAP
CORR_SAMPLE_SIZE = 2_000_000   # rows sampled for the numeric-feature correlation heatmap
                               # (the Pearson estimate is unchanged at a fraction of the memory)
# ------------------------------------------------------------------------------
```

```
# ---- Data location -----------------------------------------------------------
# This notebook processes the 72 RAW monthly BTS parquet files directly (bts_2018_01.parquet
# ... bts_2023_12.parquet) rather than a single pre-processed file -- cleaning, joining to
# airport metadata, and feature engineering all happen live in the next few cells.
try:
    import google.colab  # noqa: F401
    IN_COLAB = True
except ImportError:
    IN_COLAB = False
```

```
if IN_COLAB:
    from google.colab import drive
    drive.mount("/content/drive")
    # Project folder on Drive. The first candidate that already holds `bts_parquet` is used (any depth <= 2 unde
    # searched as a last resort); if nothing is found, MyDrive/PRT661 is created. Every other sub-folder this no
    # (checkpoints, noaa, a4_outputs) is created on demand, so a fresh Drive works without manual set-up.
    _MYDRIVE = Path("/content/drive/MyDrive")
    _CANDIDATES = [_MYDRIVE / "PRT661", _MYDRIVE / "PRT661 Project", _MYDRIVE / "PRT661_Project"]
    _hit = next((c for c in _CANDIDATES if (c / "bts_parquet").is_dir()), None)
    if _hit is None:
        _found = [p.parent for p in list(_MYDRIVE.glob("*/bts_parquet")) + list(_MYDRIVE.glob("*/*/bts_parquet")
        _hit = _found[0] if _found else next((c for c in _CANDIDATES if c.is_dir()), _CANDIDATES[0])
    DRIVE_BASE = _hit
    DRIVE_BASE.mkdir(parents=True, exist_ok=True)
    print(f"Drive project folder: {DRIVE_BASE}")
    RAW_DATA_DIR = DRIVE_BASE / "bts_parquet"
    RAW_DATA_DIR.mkdir(exist_ok=True)
    OUTPUT_DIR = Path("/content/PRT661_outputs")
else:
    # Local/non-Colab fallback -- override via env var for testing.
    RAW_DATA_DIR = Path(os.environ.get("PRT661_RAW_DATA_DIR", "./bts_parquet"))
    OUTPUT_DIR = Path(os.environ.get("PRT661_OUTPUT_DIR", "./PRT661_outputs"))

FIGURES_DIR = OUTPUT_DIR / "figures"
MODELS_DIR = OUTPUT_DIR / "models"
FIGURES_DIR.mkdir(parents=True, exist_ok=True)
MODELS_DIR.mkdir(parents=True, exist_ok=True)

existing_raw = sorted(RAW_DATA_DIR.glob("bts_*.parquet"))
print(f"Found {len(existing_raw)} raw monthly parquet file(s) in {RAW_DATA_DIR}")

# rather upload directly this session), offer the browser upload widget instead.
if IN_COLAB and len(existing_raw) == 0:
    from google.colab import files
    print(f"No bts_*.parquet files found at {RAW_DATA_DIR} -- either fix the path above and "
          f"re-run this cell, or select all 72 monthly files to upload directly now "
          f"(bts_2018_01.parquet ... bts_2023_12.parquet):")
    RAW_DATA_DIR = Path("/content")
    uploaded = files.upload()
    existing_raw = sorted(RAW_DATA_DIR.glob("bts_*.parquet"))

if len(existing_raw) < 72:
    print(f"WARNING: expected 72 monthly files (Jan 2018-Dec 2023), found {len(existing_raw)}. "
          f"Missing months will simply be absent from the analysis below.")

print(f"IN_COLAB = {IN_COLAB}")
print(f"RAW_DATA_DIR = {RAW_DATA_DIR}")
print(f"{len(existing_raw)} raw monthly file(s) ready for processing")
print(f"Outputs will be saved under: {OUTPUT_DIR.resolve()}")
```

```
Mounted at /content/drive
Drive project folder: /content/drive/MyDrive/PRT661
Found 72 raw monthly parquet file(s) in /content/drive/MyDrive/PRT661/bts_parquet
IN_COLAB = True
RAW_DATA_DIR = /content/drive/MyDrive/PRT661/bts_parquet
72 raw monthly file(s) ready for processing
Outputs will be saved under: /content/PRT661_outputs
```

1.3 Checkpointing & resume

```
FORCE_RECOMPUTE = False
CHECKPOINT_MODELS = True

if IN_COLAB:
    CKPT_DIR = DRIVE_BASE / "checkpoints"                 # created below if missing
else:
    CKPT_DIR = Path(os.environ.get("PRT661_CKPT_DIR", str(OUTPUT_DIR / "checkpoints")))
CKPT_DIR.mkdir(parents=True, exist_ok=True)

# Big artefacts that must also outlive a runtime recycle live next to the
# checkpoints rather than under the ephemeral /content OUTPUT_DIR.
FEATURES_PATH  = CKPT_DIR / "flights_features.parquet"   # DuckDB-engineered table
DF_CKPT        = CKPT_DIR / "df_engineered.parquet"      # ready-to-model dataframe
OPTUNA_DB      = CKPT_DIR / "optuna_studies.db"
OPTUNA_STORAGE = f"sqlite:///{OPTUNA_DB.as_posix()}"     # per-trial Optuna resume


def _ckpt_file(name):
```

```
    return CKPT_DIR / f"{name}.pkl"


def ckpt_exists(name):
    return (not FORCE_RECOMPUTE) and _ckpt_file(name).exists()


def ckpt_save(name, obj, compress=3):
    """Atomic write: temp file + rename, so an interrupted save never leaves a
    half-written checkpoint that a later run would load as if it were complete."""
    tmp = _ckpt_file(name).with_suffix(".pkl.part")
    joblib.dump(obj, tmp, compress=compress)
    tmp.replace(_ckpt_file(name))
    print(f"  [checkpoint saved]    {name}")
    return obj


def ckpt_load(name):
    print(f"  [checkpoint restored] {name}  (skipping recompute)")
    return joblib.load(_ckpt_file(name))


def ckpt_step(name, compute_fn, compress=3):
    """Return checkpoint `name` if present, else run compute_fn(), save, return."""
    if ckpt_exists(name):
        return ckpt_load(name)
    return ckpt_save(name, compute_fn(), compress=compress)


def df_checkpoint_exists():
    return (not FORCE_RECOMPUTE) and DF_CKPT.exists()


# ---- Resume manifest ------------------------------------------------------
print(f"Checkpoint dir: {CKPT_DIR}")
if FORCE_RECOMPUTE:
    print("FORCE_RECOMPUTE = True  ->  every step recomputes; existing checkpoints ignored.")
else:
    _found = []
    if FEATURES_PATH.exists(): _found.append("engineered feature table (flights_features.parquet)")
    if DF_CKPT.exists():       _found.append("ready-to-model dataframe (df_engineered.parquet)")
    if OPTUNA_DB.exists():     _found.append("Optuna study database (optuna_studies.db)")
    _found += [p.stem for p in sorted(CKPT_DIR.glob("*.pkl"))]
    if _found:
        print("Existing checkpoints found -- these will be restored, not recomputed:")
        for _f in _found:
            print(f"  - {_f}")
    else:
        print("No checkpoints yet -- fresh run; every step will compute and save one.")
```

```
Checkpoint dir: /content/drive/MyDrive/PRT661/checkpoints
Existing checkpoints found -- these will be restored, not recomputed:
  - engineered feature table (flights_features.parquet)
  - ready-to-model dataframe (df_engineered.parquet)
  - Optuna study database (optuna_studies.db)
  - covid_check
  - fairness
  - final_lightgbm_tuned
  - final_logistic_regression
  - final_random_forest
  - final_regressor
  - final_xgboost_tuned
  - shap
  - wf_fold2019_lightgbm
  - wf_fold2019_logistic_regression
  - wf_fold2019_random_forest
  - wf_fold2019_xgboost
  - wf_fold2022_lightgbm
  - wf_fold2022_logistic_regression
  - wf_fold2022_random_forest
  - wf_fold2022_xgboost
  - wf_fold2023_lightgbm
  - wf_fold2023_logistic_regression
  - wf_fold2023_random_forest
  - wf_fold2023_xgboost
```

```
LOCAL_RAW_DATA_DIR = Path("/content/bts_parquet")

needs_staging = IN_COLAB and str(RAW_DATA_DIR).startswith("/content/drive")

# Skip the multi-GB raw-file copy entirely when an engineered checkpoint
# already exists -- the raw files are only needed to (re)build it.
```

```
if needs_staging and (df_checkpoint_exists() or FEATURES_PATH.exists()):
    needs_staging = False
    print("Engineered-data checkpoint present -- skipping raw-file staging.")

if needs_staging:
    import shutil
    LOCAL_RAW_DATA_DIR.mkdir(parents=True, exist_ok=True)

    to_copy = []
    for src in existing_raw:
        dst = LOCAL_RAW_DATA_DIR / src.name
        src_size = src.stat().st_size
        if dst.exists() and dst.stat().st_size == src_size:
            continue
        to_copy.append((src, dst, src_size))

    total_bytes = sum(s for _, _, s in to_copy)
    already_done = len(existing_raw) - len(to_copy)
    print(f"Staging raw files to local disk: {len(to_copy)} to copy "
          f"({total_bytes/1e9:.2f} GB), {already_done} already present and up to date.")

    copied_bytes = 0
    t0 = time.time()
    for i, (src, dst, src_size) in enumerate(to_copy, 1):
        tmp_dst = dst.with_name(dst.name + ".part")
        shutil.copy2(src, tmp_dst)
        copied_size = tmp_dst.stat().st_size
        if copied_size != src_size:
            tmp_dst.unlink(missing_ok=True)
            raise IOError(
                f"Copy size mismatch for {src.name}: source {src_size:,} bytes, "
                f"copied {copied_size:,} bytes. The Drive read may have been interrupted -- "
                f"just re-run this cell (already-good files are skipped automatically)."
            )
        tmp_dst.rename(dst)
        copied_bytes += src_size
        elapsed = time.time() - t0
        rate_mbs = copied_bytes / elapsed / 1e6 if elapsed > 0 else 0
        eta_s = (total_bytes - copied_bytes) / (rate_mbs * 1e6) if rate_mbs > 0 else 0
        print(f"\r  [{i}/{len(to_copy)}] {src.name}  "
              f"{copied_bytes/1e9:.2f}/{total_bytes/1e9:.2f} GB  "
              f"{rate_mbs:.0f} MB/s  ETA {eta_s:.0f}s   ", end="", flush=True)
    if to_copy:
        print(f"\nCopied {len(to_copy)} file(s), {copied_bytes/1e9:.2f} GB, "
              f"in {time.time()-t0:.0f}s.")

    RAW_DATA_DIR = LOCAL_RAW_DATA_DIR
    existing_raw = sorted(RAW_DATA_DIR.glob("bts_*.parquet"))
else:
    print(f"Files already local at {RAW_DATA_DIR} -- no staging copy needed.")

print(f"RAW_DATA_DIR now points to local disk: {RAW_DATA_DIR}")
```

```
Engineered-data checkpoint present -- skipping raw-file staging.
Files already local at /content/drive/MyDrive/PRT661/bts_parquet -- no staging copy needed.
RAW_DATA_DIR now points to local disk: /content/drive/MyDrive/PRT661/bts_parquet
```

```
def save_fig(fig, name, tight=True):
    """Save a matplotlib figure to FIGURES_DIR as PNG (300dpi) and close it."""
    if tight:
        fig.tight_layout()
    path = FIGURES_DIR / f"{name}.png"
    fig.savefig(path, dpi=300, bbox_inches="tight")
    print(f"[saved] {path.name}")
    plt.show()
    plt.close(fig)


def save_plotly(fig, name):
    """Save a Plotly figure as both an interactive HTML and a static PNG."""
    html_path = FIGURES_DIR / f"{name}.html"
    fig.write_html(str(html_path))
    try:
        png_path = FIGURES_DIR / f"{name}.png"
        fig.write_image(str(png_path), scale=2)
        print(f"[saved] {html_path.name}, {png_path.name}")
    except Exception as exc:
        print(f"[saved] {html_path.name} (static PNG export skipped: {exc})")
    fig.show()
```

2. Load & Process Data

```
def _build_feature_table():
    # Download OpenFlights airport metadata (small, public, no auth needed).
    AIRPORTS_PATH = OUTPUT_DIR.parent / "airports.dat"
    if IN_COLAB and (DRIVE_BASE / "openflights" / "airports.dat").exists():     # use the copy kept on Drive
        AIRPORTS_PATH = DRIVE_BASE / "openflights" / "airports.dat"
    AIRPORTS_URL = "https://raw.githubusercontent.com/jpatokal/openflights/master/data/airports.dat"

    if not AIRPORTS_PATH.exists():
        import urllib.request
        urllib.request.urlretrieve(AIRPORTS_URL, AIRPORTS_PATH)
    print(f"OpenFlights airport metadata: {AIRPORTS_PATH} ({AIRPORTS_PATH.stat().st_size/1e3:.0f} KB)")

    AIRPORT_COLUMNS = [
        "airport_id", "name", "city", "country", "iata", "icao",
        "latitude", "longitude", "altitude", "timezone", "dst",
        "tz_database_timezone", "type", "source",
    ]

    def _detect_total_ram_gb():
        """Read total system RAM from /proc/meminfo (Colab and most Linux runtimes).
        Falls back to a conservative 12GB assumption (Colab free-tier default) if
        that file isn't available, e.g. on some non-Linux local setups."""
        try:
            with open("/proc/meminfo") as f:
                for line in f:
                    if line.startswith("MemTotal:"):
                        kb = int(line.split()[1])
                        return kb / 1e6
        except Exception:
            pass
        return 12.0

    t0 = time.time()
    total_ram_gb = _detect_total_ram_gb()

    duckdb_mem_limit_gb = max(2, int(total_ram_gb * 0.7))
    print(f"Detected ~{total_ram_gb:.1f} GB total RAM -- capping DuckDB at {duckdb_mem_limit_gb} GB")

    con = duckdb.connect()
    con.execute(f"PRAGMA memory_limit='{duckdb_mem_limit_gb}GB'")
    con.execute("PRAGMA threads=2")
    con.execute("PRAGMA preserve_insertion_order=false")
    duckdb_tmp_dir = OUTPUT_DIR.parent / ".duckdb_tmp"
    duckdb_tmp_dir.mkdir(parents=True, exist_ok=True)
    con.execute(f"PRAGMA temp_directory='{duckdb_tmp_dir.as_posix()}'")

    con.execute(f"""
        CREATE VIEW airports AS
        SELECT * FROM read_csv_auto('{AIRPORTS_PATH.as_posix()}', header=false, names={AIRPORT_COLUMNS})
    """)
    con.execute("""
        CREATE TABLE airports_clean AS
        SELECT DISTINCT ON (iata) iata, latitude, longitude, city, country
        FROM airports
        WHERE iata IS NOT NULL AND iata != '\\N' AND iata != ''
    """)

    raw_glob = str(RAW_DATA_DIR / "bts_*.parquet")
    con.execute(f"""
        CREATE VIEW flights_raw AS
        SELECT * FROM read_parquet('{raw_glob}', union_by_name=true)
    """)
    n_raw = con.execute("SELECT COUNT(*) FROM flights_raw").fetchone()[0]
    print(f"[load] {n_raw:,} raw flight records from {len(existing_raw)} monthly files")


    con.execute("""
        CREATE TABLE flights_base AS
        SELECT
            f.flight_date, f.carrier, f.origin, f.destination,
            f.crs_dep_time, f.distance, f.arr_del15, f.arr_delay_minutes,
            o.latitude  AS origin_lat, o.longitude AS origin_lon,
            d.latitude  AS dest_lat,   d.longitude AS dest_lon,
            LEAST(GREATEST(f.crs_dep_time // 100, 0), 23) AS crs_dep_hour,
            (isodow(f.flight_date) - 1) AS day_of_week,
            CASE WHEN (isodow(f.flight_date) - 1) IN (5, 6) THEN 1 ELSE 0 END AS is_weekend,
            month(f.flight_date) AS month,
```

```
            year(f.flight_date) AS year
        FROM (
            SELECT DISTINCT ON (FlightDate, Reporting_Airline, Flight_Number_Reporting_Airline, Origin, Dest)
                CAST(FlightDate AS DATE)               AS flight_date,
                Reporting_Airline                       AS carrier,
                Origin                                  AS origin,
                Dest                                     AS destination,
                CAST(CRSDepTime AS INTEGER)             AS crs_dep_time,
                CAST(Distance AS DOUBLE)                AS distance,
                CAST(COALESCE(ArrDel15, 0) AS INTEGER)   AS arr_del15,
                CAST(COALESCE(ArrDelayMinutes, 0) AS DOUBLE) AS arr_delay_minutes
            FROM flights_raw
            WHERE Cancelled = 0 AND Diverted = 0
              AND FlightDate IS NOT NULL AND Origin IS NOT NULL AND Dest IS NOT NULL
        ) f
        LEFT JOIN airports_clean o ON f.origin = o.iata
        LEFT JOIN airports_clean d ON f.destination = d.iata
    """)
    n_base = con.execute("SELECT COUNT(*) FROM flights_base").fetchone()[0]
    print(f"[clean+join] {n_base:,} records")

    # 7-day rolling delay rate per carrier and per origin airport, over PRIOR
    # days only (ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING) so the current day's
    # outcome cannot enter its own feature. A key's first day, with no prior
    # data, falls back to the unweighted mean of the per-day rates.
    for key in ["carrier", "origin"]:
        con.execute(f"""
            CREATE TABLE {key}_daily AS
            SELECT {key}, flight_date AS day, AVG(arr_del15) AS daily_rate
            FROM flights_base GROUP BY {key}, flight_date
        """)
        con.execute(f"""
            CREATE TABLE {key}_daily_rate AS
            SELECT {key}, day,
                AVG(daily_rate) OVER (
                    PARTITION BY {key} ORDER BY day
                    ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING
                ) AS {key}_delay_rate_7d
            FROM {key}_daily
        """)
    overall_carrier_rate = con.execute("SELECT AVG(daily_rate) FROM carrier_daily").fetchone()[0]
    overall_origin_rate = con.execute("SELECT AVG(daily_rate) FROM origin_daily").fetchone()[0]

    # Written to the module-level FEATURES_PATH (kept on Drive).
    con.execute(f"""
        COPY (
            SELECT b.*,
                COALESCE(cr.carrier_delay_rate_7d, {overall_carrier_rate}) AS carrier_delay_rate_7d,
                COALESCE(orr.origin_delay_rate_7d, {overall_origin_rate}) AS origin_delay_rate_7d
            FROM flights_base b
            LEFT JOIN carrier_daily_rate cr ON b.carrier = cr.carrier AND b.flight_date = cr.day
            LEFT JOIN origin_daily_rate orr ON b.origin = orr.origin AND b.flight_date = orr.day
            WHERE b.distance IS NOT NULL AND b.origin_lat IS NOT NULL AND b.dest_lat IS NOT NULL
            ORDER BY b.flight_date
        ) TO '{FEATURES_PATH.as_posix()}' (FORMAT PARQUET)
    """)
    n_features = con.execute(f"SELECT COUNT(*) FROM read_parquet('{FEATURES_PATH.as_posix()}')").fetchone()[0]
    con.close()
    import shutil as _shutil
    _shutil.rmtree(duckdb_tmp_dir, ignore_errors=True)
    print(f"[features] {n_features:,} fully-engineered records -> {FEATURES_PATH.name} ({time.time()-t0:.0f}s)")

if df_checkpoint_exists():
    print("Ready-to-model df checkpoint present -- skipping DuckDB feature engineering.")
elif FEATURES_PATH.exists():
    print(f"Reusing engineered table {FEATURES_PATH.name} "
          f"({FEATURES_PATH.stat().st_size / 1e6:.0f} MB) -- skipping DuckDB pipeline.")
else:
    _build_feature_table()
```

```
Ready-to-model df checkpoint present -- skipping DuckDB feature engineering.
```

```
t0 = time.time()

if df_checkpoint_exists():
    df = pd.read_parquet(DF_CKPT)
    print(f"Restored ready-to-model dataframe from checkpoint: {len(df):,} rows x "
          f"{df.shape[1]} cols in {time.time() - t0:.1f}s "
          f"({df.memory_usage(deep=True).sum() / 1e9:.2f} GB)")
else:
```

```
    import pyarrow.parquet as pq
    import pyarrow as pa

    table = pq.read_table(FEATURES_PATH)
    for col_name in ["carrier", "origin", "destination"]:
        idx = table.schema.get_field_index(col_name)
        table = table.set_column(idx, col_name, table.column(col_name).dictionary_encode())
    for col_name in ["distance", "origin_lat", "origin_lon", "dest_lat", "dest_lon",
                     "arr_delay_minutes", "carrier_delay_rate_7d", "origin_delay_rate_7d"]:
        idx = table.schema.get_field_index(col_name)
        table = table.set_column(idx, col_name, table.column(col_name).cast(pa.float32()))
    for col_name, dtype in [("crs_dep_time", pa.int16()), ("crs_dep_hour", pa.int8()),
                            ("day_of_week", pa.int8()), ("is_weekend", pa.int8()),
                            ("month", pa.int8()), ("year", pa.int16()), ("arr_del15", pa.int8())]:
        idx = table.schema.get_field_index(col_name)
        table = table.set_column(idx, col_name, table.column(col_name).cast(dtype))


    df = table.to_pandas(date_as_object=False, split_blocks=True, self_destruct=True)
    del table
    gc.collect()


    us_holidays = holidays.UnitedStates(years=df["flight_date"].dt.year.unique().tolist())
    holiday_dates = pd.to_datetime(list(us_holidays.keys()))
    df["is_holiday"] = df["flight_date"].dt.normalize().isin(holiday_dates).astype("int8")

    print(f"Loaded + engineered {len(df):,} rows x {df.shape[1]} columns in {time.time()-t0:.1f}s")
    print(f"Memory footprint: {df.memory_usage(deep=True).sum() / 1e9:.2f} GB")
    df.head()
print(f"df ready: {len(df):,} rows, {df.shape[1]} columns, {df.memory_usage(deep=True).sum() / 1e9:.2f} GB in me
```

```
Restored ready-to-model dataframe from checkpoint: 37,877,189 rows x 22 cols in 9.0s (2.27 GB)
df ready: 37,877,189 rows, 22 columns, 2.27 GB in memory
```

3. Five Vs Analysis (measured, not estimated)

Assessment 1 estimated these figures ahead of time; here they are recomputed directly from the loaded dataset.

```
five_vs = {
    "Volume": f"{len(df):,} flight records, {df.memory_usage(deep=True).sum()/1e9:.2f} GB in memory "
              f"({sum(p.stat().st_size for p in existing_raw)/1e9:.2f} GB raw Parquet input "
              f"across {len(existing_raw)} monthly files)",
    "Velocity": f"Batch monthly ingestion, {df['flight_date'].min().date()} to {df['flight_date'].max().date()}
                f"({df['year'].nunique()} years, {df.groupby(['year', df['flight_date'].dt.month]).ngroups} mont
    "Variety": f"{df['carrier'].nunique()} carriers, {df['origin'].nunique()} origin airports, "
               f"{df['destination'].nunique()} destination airports",
    "Veracity": f"{df.isna().sum().sum()} missing values after upstream cleaning "
                f"(dedup on flight-leg key, cancelled/diverted flights dropped)",
    "Value": f"Overall delay rate (15+ min): {df['arr_del15'].mean()*100:.1f}% -- "
             f"supports delay classification, duration regression, carrier/airport benchmarking",
}
for k, v in five_vs.items():
    print(f"{k:10s}: {v}")

pd.DataFrame(five_vs.items(), columns=["Dimension", "Measured value"]).to_csv(
    OUTPUT_DIR / "five_vs_summary.csv", index=False
)
```

```
Volume    : 37,877,189 flight records, 2.27 GB in memory (1.30 GB raw Parquet input across 72 monthly files)
Velocity  : Batch monthly ingestion, 2018-01-01 to 2023-12-31 (6 years, 72 months)
Variety   : 19 carriers, 379 origin airports, 379 destination airports
Veracity  : 0 missing values after upstream cleaning (dedup on flight-leg key, cancelled/diverted flights dropped
Value     : Overall delay rate (15+ min): 18.3% -- supports delay classification, duration regression, carrier/ai
```

4. Graph-Derived Hub Features

```
if {"origin_degree_centrality", "origin_hub_tier"}.issubset(df.columns):
    print("Graph-derived hub features already present (restored from checkpoint).")
else:
    # Out-degree: distinct destinations reachable from each origin airport.
    degree = df.groupby("origin", observed=True)["destination"].nunique()
    degree_norm = (degree - degree.min()) / (degree.max() - degree.min())
    df["origin_degree_centrality"] = df["origin"].map(degree_norm).astype("float32")
```

```
    # Hub tier: by total flight volume out of each origin airport.
    volume = df["origin"].value_counts()
    top_decile = volume.quantile(0.90)
    top_4deciles = volume.quantile(0.60)

    def tier_for(v):
        if v >= top_decile:
            return "major"
        elif v >= top_4deciles:
            return "medium"
        return "small"

    hub_tier_map = volume.apply(tier_for)
    df["origin_hub_tier"] = df["origin"].map(hub_tier_map).astype("category")

    print(df[["origin", "origin_degree_centrality", "origin_hub_tier"]].drop_duplicates("origin").shape[0], "air
    print(df["origin_hub_tier"].value_counts())
    df[["origin", "origin_degree_centrality", "origin_hub_tier"]].drop_duplicates("origin").sort_values(
        "origin_degree_centrality", ascending=False
    ).head(10)


# ---- Persist the fully-engineered dataframe ---------------------------------
# Cache the assembled dataset so a later 'Run all' reloads it instead of
# rebuilding from the 72 raw monthly files.
if not df_checkpoint_exists():
    try:
        _t = time.time()
        df.to_parquet(DF_CKPT, compression="zstd", index=False)
        print(f"  [checkpoint saved]    {DF_CKPT.name} "
              f"({DF_CKPT.stat().st_size / 1e6:.0f} MB, {time.time() - _t:.0f}s)")
    except Exception as _exc:
        print(f"  WARNING: could not write {DF_CKPT.name} ({_exc}). Continuing without "
              f"dataframe-level resume; the feature-table + per-model checkpoints still apply.")
```

```
Graph-derived hub features already present (restored from checkpoint).
```

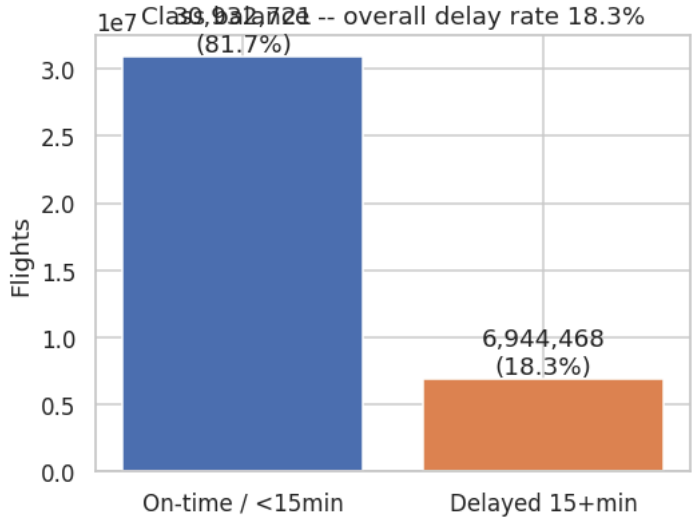
5. Exploratory Data Analysis (Advanced Visualisations)

Every figure in this section is saved to `figures/` as it's produced.

5.1 Overall delay rate and class balance

```
fig, ax = plt.subplots(figsize=(5, 4))
counts = df["arr_del15"].value_counts().sort_index()
labels = ["On-time / <15min", "Delayed 15+min"]
colors = ["#4C72B0", "#DD8452"]
ax.bar(labels, counts.values, color=colors)
for i, v in enumerate(counts.values):
    ax.text(i, v + len(df)*0.01, f"{v:,}\n({v/len(df)*100:.1f}%)", ha="center")
ax.set_ylabel("Flights")
ax.set_title(f"Class balance -- overall delay rate {df['arr_del15'].mean()*100:.1f}%")
save_fig(fig, "01_class_balance")
```

```
[saved] 01_class_balance.png
```



5.2 Monthly delay rate trend, 2018-2023 (COVID period highlighted)

```
monthly = df.groupby(df["flight_date"].dt.to_period("M"))["arr_del15"].mean() * 100
monthly.index = monthly.index.to_timestamp()

fig, ax = plt.subplots(figsize=(13, 5))
ax.plot(monthly.index, monthly.values, marker="o", markersize=3, color="#4C72B0")
ax.axvspan(pd.Timestamp("2020-01-01"), pd.Timestamp("2021-12-31"), color="grey", alpha=0.15,
           label="COVID-19 period (2020-2021)")
ax.set_ylabel("Delay rate (%)")
ax.set_title("Monthly flight delay rate, Jan 2018 - Dec 2023")
ax.legend()
save_fig(fig, "02_monthly_delay_trend")
```

```
[saved] 02_monthly_delay_trend.png
```

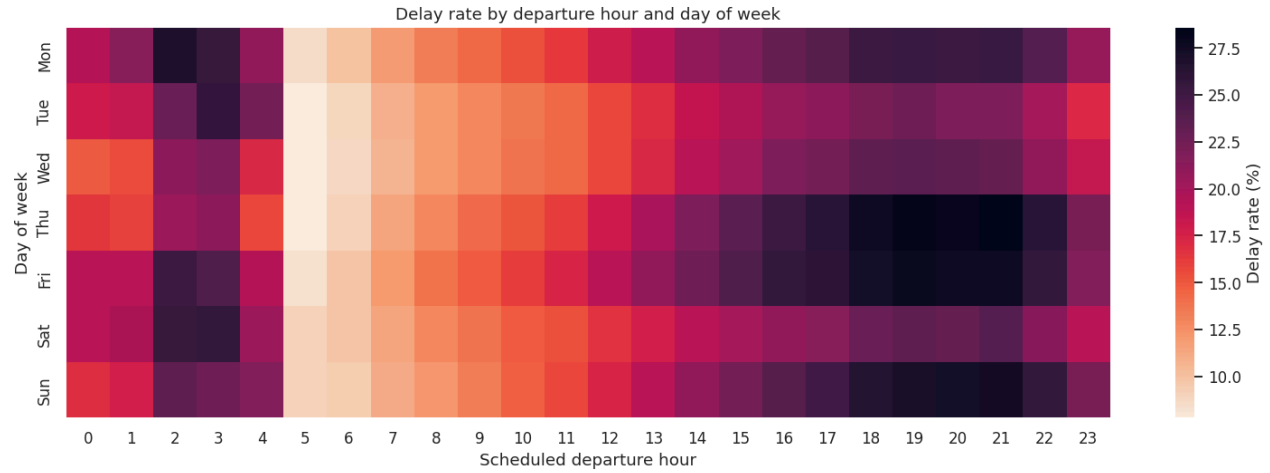


5.3 Delay rate by hour-of-day x day-of-week (heatmap)

```
pivot = df.pivot_table(index="day_of_week", columns="crs_dep_hour", values="arr_del15", aggfunc="mean") * 100
day_labels = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

fig, ax = plt.subplots(figsize=(14, 5))
sns.heatmap(pivot, cmap="rocket_r", ax=ax, cbar_kws={"label": "Delay rate (%)"},
            yticklabels=day_labels)
ax.set_xlabel("Scheduled departure hour")
ax.set_ylabel("Day of week")
ax.set_title("Delay rate by departure hour and day of week")
save_fig(fig, "03_hour_dow_heatmap")
```

```
[saved] 03_hour_dow_heatmap.png
```

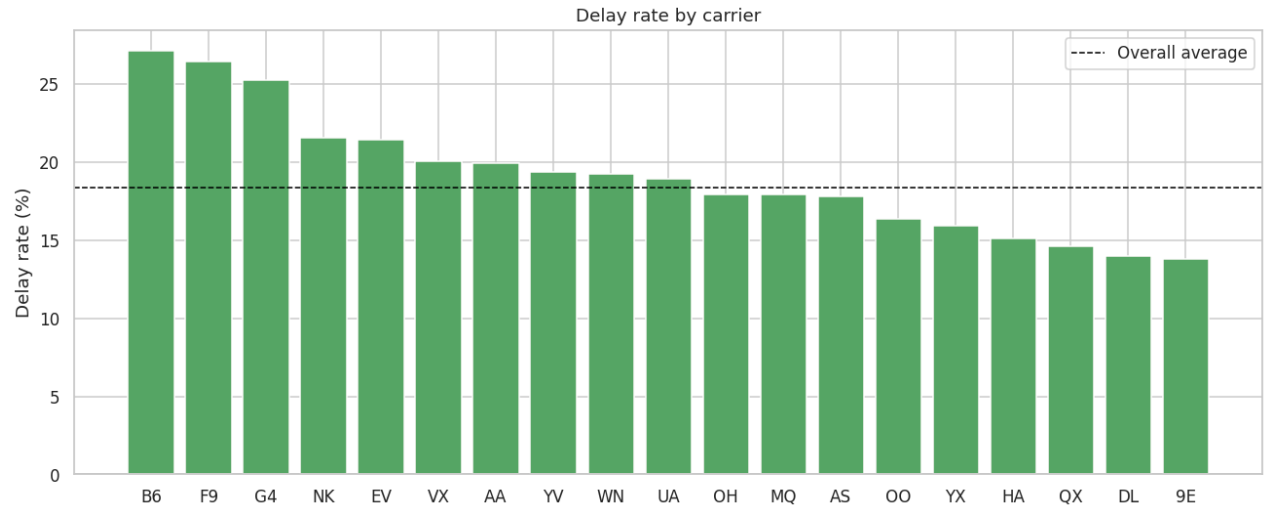


5.4 Delay rate by carrier

```
carrier_rate = (df.groupby("carrier", observed=True)["arr_del15"].mean() * 100).sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(12, 5))
bars = ax.bar(carrier_rate.index.astype(str), carrier_rate.values, color="#55A868")
ax.axhline(df["arr_del15"].mean()*100, color="black", linestyle="--", linewidth=1, label="Overall average")
ax.set_ylabel("Delay rate (%)")
ax.set_title("Delay rate by carrier")
ax.legend()
save_fig(fig, "04_delay_rate_by_carrier")
```

```
[saved] 04_delay_rate_by_carrier.png
```



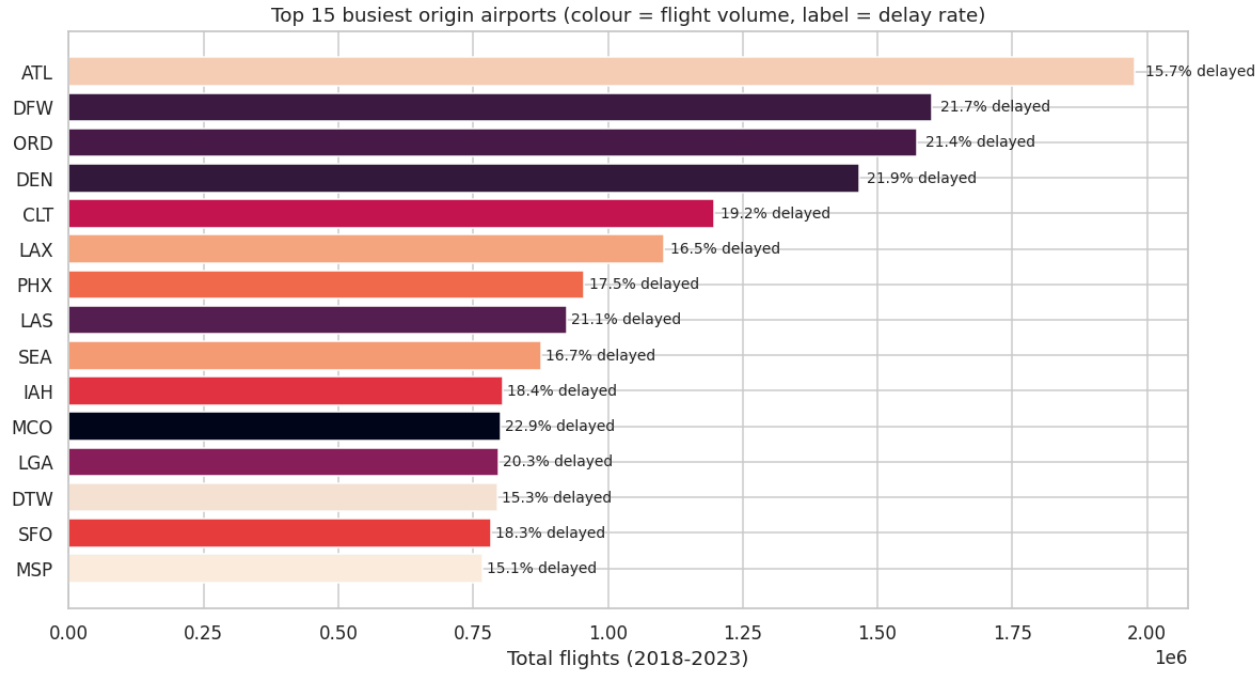
5.5 Top 15 origin airports by volume, coloured by delay rate

```
airport_stats = df.groupby("origin", observed=True).agg(
    flights=("arr_del15", "size"), delay_rate=("arr_del15", "mean")
).sort_values("flights", ascending=False).head(15)
```

```
airport_stats["delay_rate"] *= 100

fig, ax = plt.subplots(figsize=(11, 6))
norm = plt.Normalize(airport_stats["delay_rate"].min(), airport_stats["delay_rate"].max())
colors = plt.get_cmap("rocket_r")(norm(airport_stats["delay_rate"]))
bars = ax.barh(airport_stats.index[::-1], airport_stats["flights"][::-1], color=colors[::-1])
for bar, rate in zip(bars, airport_stats["delay_rate"][::-1]):
    ax.text(bar.get_width()*1.01, bar.get_y()+bar.get_height()/2, f"{rate:.1f}% delayed",
            va="center", fontsize=9)
ax.set_xlabel("Total flights (2018–2023)")
ax.set_title("Top 15 busiest origin airports (colour = flight volume, label = delay rate)")
save_fig(fig, "05_top_airports_volume_delay")
```

```
[saved] 05_top_airports_volume_delay.png
```

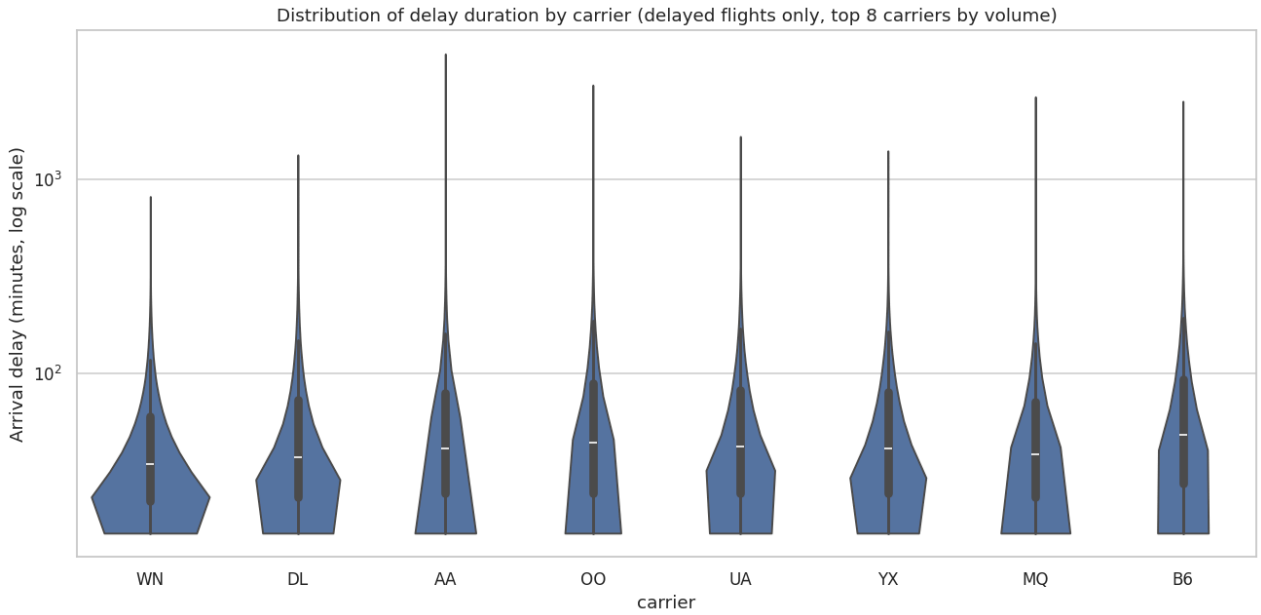


5.6 Delay-minute distribution by carrier (delayed flights only, log scale)

```
delayed_only = df[df["arr_del15"] == 1]
top8_carriers = df["carrier"].value_counts().head(8).index.tolist()

fig, ax = plt.subplots(figsize=(12, 6))
sns.violinplot(
    data=delayed_only[delayed_only["carrier"].isin(top8_carriers)],
    x="carrier", y="arr_delay_minutes", ax=ax, cut=0, order=top8_carriers,
)
ax.set_yscale("log")
ax.set_ylabel("Arrival delay (minutes, log scale)")
ax.set_title("Distribution of delay duration by carrier (delayed flights only, top 8 carriers by volume)")
save_fig(fig, "06_delay_duration_violin_by_carrier")
```

```
[saved] 06_delay_duration_violin_by_carrier.png
```

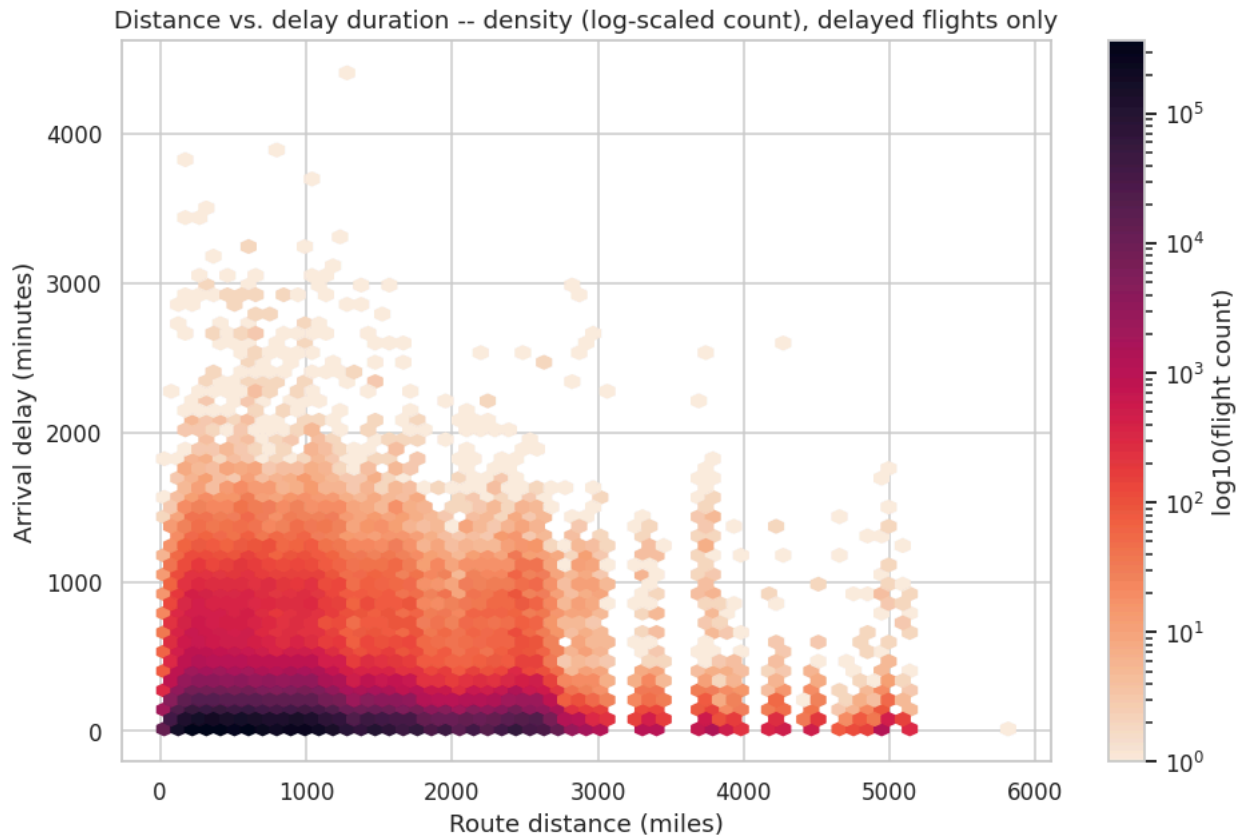


5.7 Distance vs. delay minutes (hexbin density -- 37M points can't be scatter-plotted directly)

```
fig, ax = plt.subplots(figsize=(9, 6))
hb = ax.hexbin(delayed_only["distance"], delayed_only["arr_delay_minutes"],
               gridsize=60, cmap="rocket_r", mincnt=1, bins="log")
ax.set_xlabel("Route distance (miles)")
ax.set_ylabel("Arrival delay (minutes)")
ax.set_title("Distance vs. delay duration -- density (log-scaled count), delayed flights only")
fig.colorbar(hb, ax=ax, label="log10(flight count)")
save_fig(fig, "07_distance_vs_delay_hexbin")

# delayed_only (a ~15-20% full-row copy of df, made back in 5.6) isn't used
# again anywhere below -- freeing it here instead of leaving it resident for
# the rest of the notebook.
del delayed_only
gc.collect()
```

```
[saved] 07_distance_vs_delay_hexbin.png
```




```
16668
```

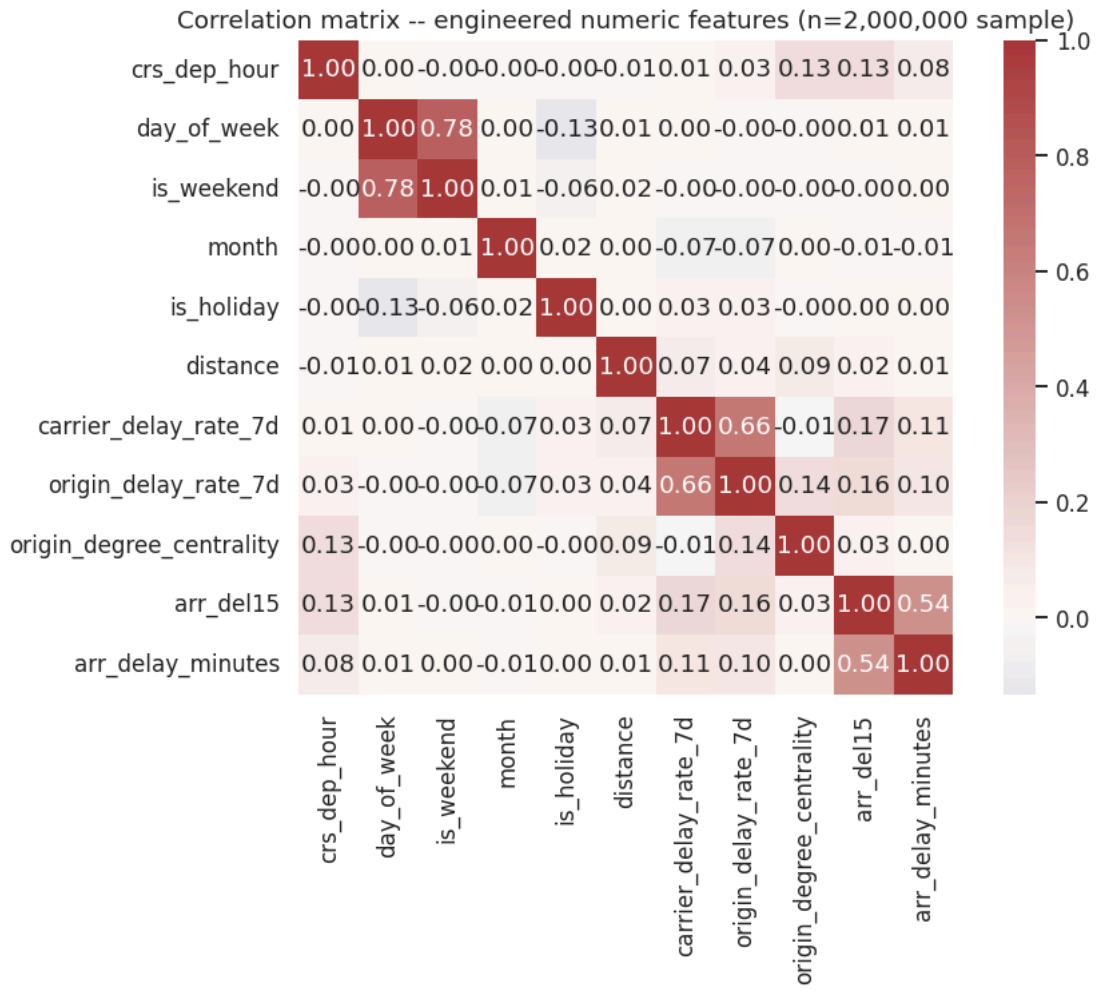
5.8 Correlation heatmap of engineered numeric features

```
numeric_feats = ["crs_dep_hour", "day_of_week", "is_weekend", "month", "is_holiday",
                 "distance", "carrier_delay_rate_7d", "origin_delay_rate_7d",
                 "origin_degree_centrality", "arr_del15", "arr_delay_minutes"]

# Sampled rather than run on all 37.9M rows (see CORR_SAMPLE_SIZE in the Config
# cell); the Pearson estimate is unchanged at a fraction of the memory.
n_corr_sample = min(CORR_SAMPLE_SIZE, len(df))
corr_sample = df[numeric_feats].sample(n=n_corr_sample, random_state=RANDOM_STATE)
corr = corr_sample.corr()
del corr_sample
gc.collect()

fig, ax = plt.subplots(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="vlag", center=0, ax=ax, square=True)
ax.set_title(f"Correlation matrix -- engineered numeric features (n={n_corr_sample:,} sample)")
save_fig(fig, "08_correlation_heatmap")
```

```
[saved] 08_correlation_heatmap.png
```

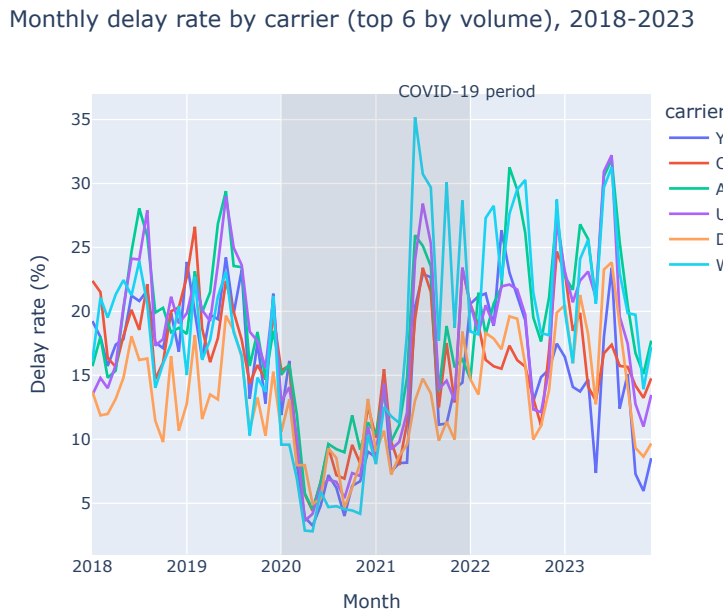


5.9 Interactive: monthly delay rate for the top 6 carriers (Plotly)

```
top6 = df["carrier"].value_counts().head(6).index.tolist()
# Aggregate first (output is only months x carriers), then filter to the top 6;
# filtering the 37.9M-row frame before aggregating would copy most of it.
monthly_carrier_all = (
    df.groupby([df["flight_date"].dt.to_period("M"), "carrier"], observed=True)["arr_del15"]
    .mean().mul(100).rename("delay_rate").reset_index()
)
monthly_carrier = monthly_carrier_all[monthly_carrier_all["carrier"].isin(top6)].copy()
del monthly_carrier_all
monthly_carrier["flight_date"] = monthly_carrier["flight_date"].dt.to_timestamp()

fig = px.line(monthly_carrier, x="flight_date", y="delay_rate", color="carrier",
              title="Monthly delay rate by carrier (top 6 by volume), 2018-2023",
              labels={"delay_rate": "Delay rate (%)", "flight_date": "Month"})
fig.add_vrect(x0="2020-01-01", x1="2021-12-31", fillcolor="grey", opacity=0.15,
              line_width=0, annotation_text="COVID-19 period")
save_plotly(fig, "09_interactive_monthly_delay_by_carrier")
```

```
[saved] 09_interactive_monthly_delay_by_carrier.html (static PNG export skipped:
Image export using the "kaleido" engine requires the kaleido package,
which can be installed using pip:
    $ pip install -U kaleido
)
```

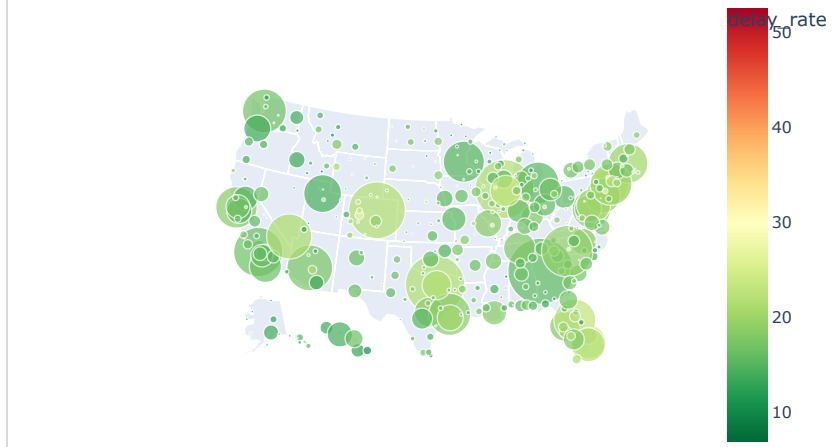


5.10 Interactive: US map of origin airports (Plotly geo scatter)

```
airport_geo = df.groupby("origin", observed=True).agg(
    flights=("arr_del15", "size"), delay_rate=("arr_del15", "mean"),
    lat=("origin_lat", "first"), lon=("origin_lon", "first"),
).reset_index()
airport_geo["delay_rate"] *= 100
airport_geo = airport_geo[airport_geo["flights"] >= 100]  # drop near-empty airports for a legible map

fig = px.scatter_geo(
    airport_geo, lat="lat", lon="lon", size="flights", color="delay_rate",
    hover_name="origin", scope="usa", color_continuous_scale="RdYlGn_r",
    size_max=35, title="Origin airports: bubble size = flight volume, colour = delay rate (%)",
)
save_plotly(fig, "10_interactive_us_airport_map")
```

```
[saved] 10_interactive_us_airport_map.html (static PNG export skipped:
Image export using the "kaleido" engine requires the kaleido package,
which can be installed using pip:
    $ pip install -U kaleido
)
```



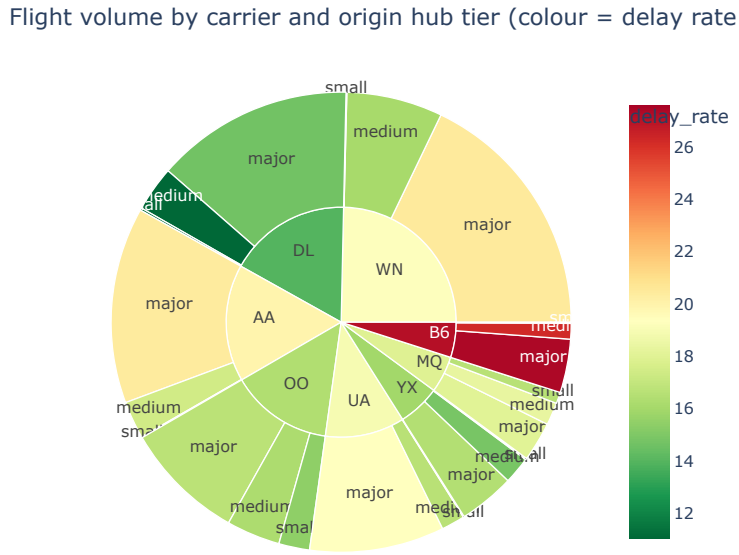
5.11 Interactive: carrier -> hub tier delay contribution (Plotly sunburst)

```
sun_data = df.groupby(["carrier", "origin_hub_tier"], observed=True).agg(
    flights=("arr_del15", "size"), delay_rate=("arr_del15", "mean")
).reset_index()
sun_data = sun_data[sun_data["carrier"].isin(top8_carriers)].copy()
sun_data["delay_rate"] *= 100

# Cast the two category columns to plain strings: px.sunburst groups without
# observed=True and would otherwise include empty categories, whose zero
# weight triggers a ZeroDivisionError in its colour averaging.
sun_data["carrier"] = sun_data["carrier"].astype(str)
sun_data["origin_hub_tier"] = sun_data["origin_hub_tier"].astype(str)

fig = px.sunburst(
    sun_data, path=["carrier", "origin_hub_tier"], values="flights", color="delay_rate",
    color_continuous_scale="RdYlGn_r", title="Flight volume by carrier and origin hub tier (colour = delay rate
)
save_plotly(fig, "11_interactive_sunburst_carrier_hub")
```

```
[saved] 11_interactive_sunburst_carrier_hub.html (static PNG export skipped:
Image export using the "kaleido" engine requires the kaleido package,
which can be installed using pip:
    $ pip install -U kaleido
)
```



6. Modelling Setup -- Walk-Forward Cross-Validation

```
NUMERIC_FEATURES = [
    "crs_dep_hour", "day_of_week", "is_weekend", "month", "is_holiday",
    "distance", "carrier_delay_rate_7d", "origin_delay_rate_7d", "origin_degree_centrality",
]
CATEGORICAL_FEATURES = ["carrier", "origin_hub_tier"]
ALL_FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES
TARGET = "arr_del15"

WALK_FORWARD_FOLDS = [
    {"name": "Fold 1", "train_years": [2018], "test_year": 2019},
    {"name": "Fold 2", "train_years": [2018, 2019], "test_year": 2022},
    {"name": "Fold 3 (production)", "train_years": [2018, 2019, 2022], "test_year": 2023},
]

def _gpu_available():
    """True if an NVIDIA GPU is visible to this runtime. Used to opt XGBoost
    into GPU training when one is attached, falling back to CPU otherwise."""
    try:
        import subprocess
        subprocess.run(["nvidia-smi"], stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, check=True)
        return True
    except Exception:
        return False

# XGBoost uses the GPU when one is present; the other models stay on CPU
# (scikit-learn has no GPU path; LightGBM's GPU build is not installed here).
XGB_DEVICE = "cuda" if _gpu_available() else "cpu"
XGB_TREE_METHOD = "hist"  # same tree_method value for both CPU and GPU in xgboost>=2.0; `device` picks the back
print(f"XGBoost device: {XGB_DEVICE}")

# Pin numeric model inputs to float32; a mixed int64/float32 block would be
# upcast to float64 by ColumnTransformer, doubling the matrix's memory.
for _col in NUMERIC_FEATURES:
    df[_col] = df[_col].astype("float32")

def make_preprocessor():
    # StandardScaler: needed for the logistic-regression saga solver, harmless
    # for the tree models. sparse_output=False keeps the transformed matrix
    # dense and avoids a sparse-to-CSC copy inside RandomForest.fit().
    return ColumnTransformer(
        [
            ("cat", OneHotEncoder(handle_unknown="ignore", dtype=np.float32, sparse_output=False), CATEGORICAL_F
            ("num", StandardScaler(), NUMERIC_FEATURES),
        ],
```

```
    )

def get_split(train_years, test_year):
    train = df[df["year"].isin(train_years)]
    test = df[df["year"] == test_year]
    X_train, y_train = train[ALL_FEATURES], train[TARGET]
    X_test, y_test = test[ALL_FEATURES], test[TARGET]
    return X_train, y_train, X_test, y_test

def evaluate_classifier(model, X_test, y_test):
    proba = model.predict_proba(X_test)[:, 1]
    pred = (proba >= 0.5).astype(int)
    return {
        "auc_roc": roc_auc_score(y_test, proba),
        "f1": f1_score(y_test, pred),
        "precision": precision_score(y_test, pred, zero_division=0),
        "recall": recall_score(y_test, pred),
        "brier": brier_score_loss(y_test, proba),
    }, proba, pred

print("Walk-forward folds:")
for f in WALK_FORWARD_FOLDS:
    print(f"  {f['name']}: train={f['train_years']} -> test={f['test_year']}")
```

```
XGBoost device: cuda
Walk-forward folds:
  Fold 1: train=[2018] -> test=2019
  Fold 2: train=[2018, 2019] -> test=2022
  Fold 3 (production): train=[2018, 2019, 2022] -> test=2023
```

7. Walk-Forward Cross-Validated Training -- Baseline & Advanced Models

```
def build_model(name):
    if name == "logistic_regression":
        # solver="saga": stochastic, does not hold the full design matrix in
        # memory, and scales to this row count; solves the same L2 problem.
        return Pipeline([("pre", make_preprocessor()),
                         ("model", LogisticRegression(max_iter=1000, class_weight="balanced", solver="saga"))])
    if name == "random_forest":
        return Pipeline([("pre", make_preprocessor()),
                         ("model", RandomForestClassifier(
                             n_estimators=RF_N_ESTIMATORS, max_depth=14,
                             class_weight="balanced_subsample", n_jobs=-1, random_state=RANDOM_STATE))])
    if name == "xgboost":
        return None  # built per-fold below (needs scale_pos_weight from y_train)
    if name == "lightgbm":
        return Pipeline([("pre", make_preprocessor()),
                         ("model", LGBMClassifier(
                             n_estimators=XGB_LGBM_N_ESTIMATORS, learning_rate=0.08,
                             class_weight="balanced", n_jobs=-1, random_state=RANDOM_STATE, verbosity=-1))])
    raise ValueError(name)

# Each (fold, model) fit is checkpointed: its metrics row and (y_test, proba)
# arrays are stored, not the fitted pipeline. A re-run reloads finished
# combinations and fits only the missing ones.
walk_forward_results = []
fold_predictions = {}

for fold in WALK_FORWARD_FOLDS:
    print(f"\n=== {fold['name']}: train={fold['train_years']} -> test={fold['test_year']} ===")
    fold_predictions[fold["name"]] = {}
    fold_tag = fold["name"].split()[0].lower() + str(fold["test_year"])
    split = None  # materialised lazily, only if a model in this fold needs fitting

    for model_name in ["logistic_regression", "random_forest", "xgboost", "lightgbm"]:
        ck = f"wf_{fold_tag}_{model_name}"
        if ckpt_exists(ck):
            rec = ckpt_load(ck)
        else:
            if split is None:
                split = get_split(fold["train_years"], fold["test_year"])
                print(f"  train={len(split[0]):,} rows, test={len(split[2]):,} rows")
            X_train, y_train, X_test, y_test = split
            t0 = time.time()
            if model_name == "xgboost":
                scale_pos_weight = (y_train == 0).sum() / max((y_train == 1).sum(), 1)
                model = Pipeline([("pre", make_preprocessor()),
                                  ("model", XGBClassifier(
                                      n_estimators=XGB_LGBM_N_ESTIMATORS, max_depth=6, learning_rate=0.08,
                                      scale_pos_weight=scale_pos_weight, eval_metric="auc",
```

```
                                    tree_method=XGB_TREE_METHOD, device=XGB_DEVICE,
                                    n_jobs=-1, random_state=RANDOM_STATE))])
            else:
                model = build_model(model_name)

            model.fit(X_train, y_train)
            metrics, proba, pred = evaluate_classifier(model, X_test, y_test)
            metrics.update({"fold": fold["name"], "model": model_name, "train_rows": len(X_train),
                            "test_rows": len(X_test), "seconds": round(time.time() - t0, 1)})
            rec = {"metrics": metrics,
                   "y_test": np.asarray(y_test.values),
                   "proba": np.asarray(proba)}
            ckpt_save(ck, rec)
            del model, pred
            gc.collect()

        walk_forward_results.append(rec["metrics"])
        fold_predictions[fold["name"]][model_name] = (rec["y_test"], rec["proba"])
        m = rec["metrics"]
        print(f"  [{model_name:20s}] AUC={m['auc_roc']:.4f} F1={m['f1']:.4f} ({m['seconds']:.0f}s)")

walk_forward_df = pd.DataFrame(walk_forward_results)
walk_forward_df.to_csv(OUTPUT_DIR / "walk_forward_results.csv", index=False)
walk_forward_df
```

```
=== Fold 1: train=[2018] -> test=2019 ===
  [checkpoint restored] wf_fold2019_logistic_regression  (skipping recompute)
  [logistic_regression ] AUC=0.6368 F1=0.3641 (145s)
  [checkpoint restored] wf_fold2019_random_forest  (skipping recompute)
  [random_forest       ] AUC=0.6500 F1=0.3706 (592s)
  [checkpoint restored] wf_fold2019_xgboost  (skipping recompute)
  [xgboost             ] AUC=0.6481 F1=0.3694 (31s)
  [checkpoint restored] wf_fold2019_lightgbm  (skipping recompute)
  [lightgbm            ] AUC=0.6489 F1=0.3706 (72s)

=== Fold 2: train=[2018, 2019] -> test=2022 ===
  [checkpoint restored] wf_fold2022_logistic_regression  (skipping recompute)
  [logistic_regression ] AUC=0.6518 F1=0.3980 (277s)
  [checkpoint restored] wf_fold2022_random_forest  (skipping recompute)
  [random_forest       ] AUC=0.6633 F1=0.4044 (1190s)
  [checkpoint restored] wf_fold2022_xgboost  (skipping recompute)
  [xgboost             ] AUC=0.6648 F1=0.4058 (50s)
  [checkpoint restored] wf_fold2022_lightgbm  (skipping recompute)
  [lightgbm            ] AUC=0.6657 F1=0.4061 (112s)

=== Fold 3 (production): train=[2018, 2019, 2022] -> test=2023 ===
  [checkpoint restored] wf_fold2023_logistic_regression  (skipping recompute)
  [logistic_regression ] AUC=0.6594 F1=0.3971 (450s)
  [checkpoint restored] wf_fold2023_random_forest  (skipping recompute)
  [random_forest       ] AUC=0.6730 F1=0.4052 (1798s)
  [checkpoint restored] wf_fold2023_xgboost  (skipping recompute)
  [xgboost             ] AUC=0.6739 F1=0.4057 (70s)
  [checkpoint restored] wf_fold2023_lightgbm  (skipping recompute)
  [lightgbm            ] AUC=0.6737 F1=0.4056 (156s)
```

	auc_roc	f1	precision	recall	brier	fold	model	train_rows	test_rows	seconds
0	0.636763	0.364102	0.261583	0.598772	0.236135	Fold 1	logistic_regression	7071325	7266528	144.8
1	0.649968	0.370566	0.269857	0.591198	0.227916	Fold 1	random_forest	7071325	7266528	592.2
2	0.648115	0.369435	0.270058	0.584535	0.227729	Fold 1	xgboost	7071325	7266528	31.3
3	0.648867	0.370557	0.270761	0.586855	0.228142	Fold 1	lightgbm	7071325	7266528	72.1
4	0.651800	0.398016	0.284764	0.660834	0.244558	Fold 2	logistic_regression	14337853	6528008	277.3
5	0.663289	0.404431	0.293206	0.651617	0.235228	Fold 2	random_forest	14337853	6528008	1190.2
6	0.664846	0.405780	0.295035	0.649627	0.235033	Fold 2	xgboost	14337853	6528008	49.7
7	0.665743	0.406124	0.294566	0.653689	0.235845	Fold 2	lightgbm	14337853	6528008	111.8
8	0.659378	0.397110	0.288441	0.637155	0.236690	Fold 3 (production)	logistic_regression	20865861	6740139	449.8
9	0.672976	0.405155	0.295906	0.642286	0.231686	Fold 3 (production)	random_forest	20865861	6740139	1797.5
10	0.673869	0.405668	0.296618	0.641517	0.231559		xgboost	20865861	6740139	69.7

Next steps: New interactive sheet

```
summary = walk_forward_df.groupby("model")[["auc_roc", "f1", "precision", "recall", "brier"]].agg(["mean", "std"
print("Walk-forward performance summary (mean +/- std across folds):")
```

```
summary
```

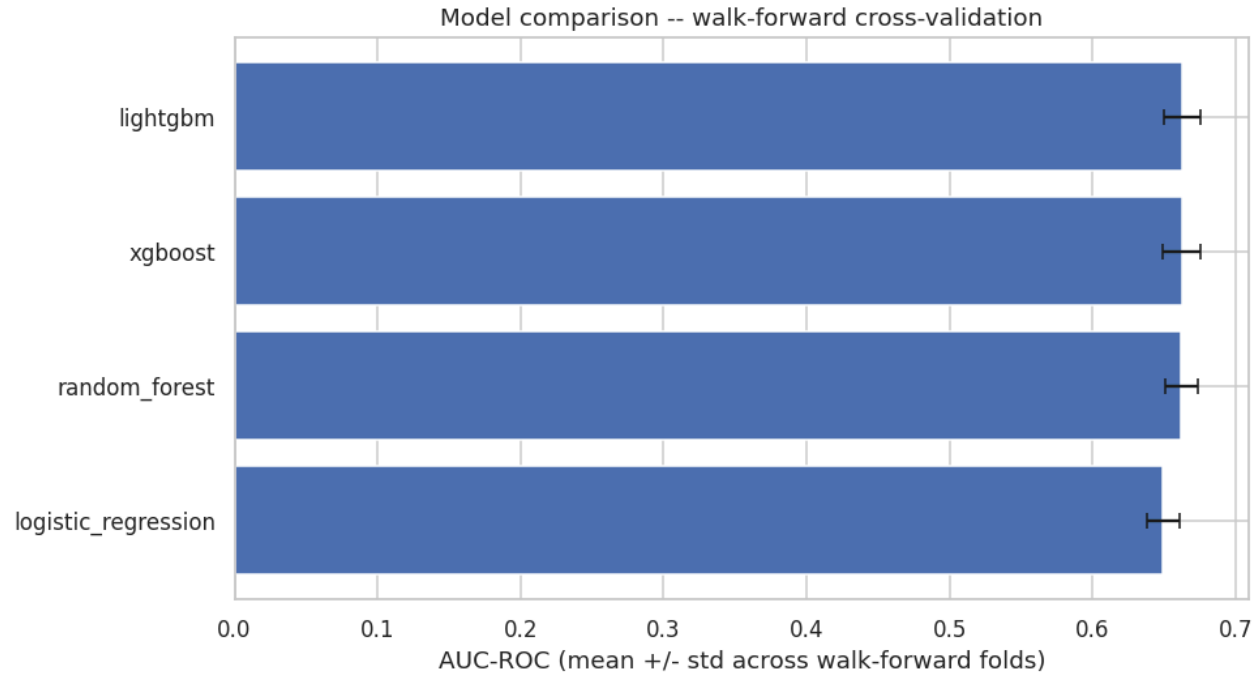
```
Walk-forward performance summary (mean +/- std across folds):
```

	auc_roc		f1		precision		recall		brier	
	mean	std	mean	std	mean	std	mean	std	mean	std
model										
lightgbm	0.662786	0.012701	0.394102	0.020393	0.287145	0.014210	0.628085	0.036053	0.232041	0.003852
logistic_regression	0.649314	0.011510	0.386409	0.019324	0.278263	0.014562	0.632254	0.031320	0.239128	0.004711
random_forest	0.662078	0.011552	0.393384	0.019764	0.286323	0.014324	0.628367	0.032526	0.231610	0.003656
xgboost	0.662276	0.013068	0.393628	0.020952	0.287237	0.014899	0.625226	0.035472	0.231441	0.003654

Next steps: New interactive sheet

```
fig, ax = plt.subplots(figsize=(9, 5))
means = walk_forward_df.groupby("model")["auc_roc"].mean().sort_values()
stds = walk_forward_df.groupby("model")["auc_roc"].std().reindex(means.index)
ax.barh(means.index, means.values, xerr=stds.values, color="#4C72B0", capsize=4)
ax.set_xlabel("AUC-ROC (mean +/- std across walk-forward folds)")
ax.set_title("Model comparison -- walk-forward cross-validation")
save_fig(fig, "12_walk_forward_model_comparison")
```

```
[saved] 12_walk_forward_model_comparison.png
```



8. Hyperparameter Tuning (Optuna) -- Production Fold

```
X_train_prod, y_train_prod, X_test_prod, y_test_prod = get_split(
    WALK_FORWARD_FOLDS[2]["train_years"], WALK_FORWARD_FOLDS[2]["test_year"]
)

# Internal validation split: last ~15% of the production training window by date.
prod_train_dates = df[df["year"].isin(WALK_FORWARD_FOLDS[2]["train_years"])]["flight_date"]
val_cutoff = prod_train_dates.quantile(0.85)
train_mask = df["year"].isin(WALK_FORWARD_FOLDS[2]["train_years"]) & (df["flight_date"] < val_cutoff)
val_mask = df["year"].isin(WALK_FORWARD_FOLDS[2]["train_years"]) & (df["flight_date"] >= val_cutoff)

X_tr_inner, y_tr_inner = df.loc[train_mask, ALL_FEATURES], df.loc[train_mask, TARGET]
X_val_inner, y_val_inner = df.loc[val_mask, ALL_FEATURES], df.loc[val_mask, TARGET]
print(f"Inner train: {len(X_tr_inner):,} rows | Inner val: {len(X_val_inner):,} rows "
      f"(cutoff date {val_cutoff.date()})")
```

```
Inner train: 17,718,989 rows | Inner val: 3,146,872 rows (cutoff date 2022-07-11)
```

```
def _fit_and_score(model, X_tr, y_tr, X_val, y_val, metric_fn, proba=True):
    """Fit + score one Optuna trial's Pipeline, then explicitly release the
    fitted pipeline and prediction array before returning. Mirrors the
    del/gc.collect() cleanup already applied to the walk-forward CV loop --
    each of the 70 HPO trials (25 XGBoost + 25 LightGBM + 20 duration
    regressor) fits a fresh Pipeline against the production fold's inner
    train split, with no cleanup between trials otherwise."""
    pred = None
    try:
        model.fit(X_tr, y_tr)
        pred = model.predict_proba(X_val)[:, 1] if proba else model.predict(X_val)
        return metric_fn(y_val, pred)
    finally:
        del model, pred
        gc.collect()


def _gc_after_trial(study, trial):
    """Optuna callback: force a gc pass after every trial (including failed
    or pruned ones), independent of the per-trial cleanup above."""
    gc.collect()


def objective_xgb(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 150, 500),
        "max_depth": trial.suggest_int("max_depth", 4, 10),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2, log=True),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6, 1.0),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-3, 10, log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-3, 10, log=True),
    }
    scale_pos_weight = (y_tr_inner == 0).sum() / max((y_tr_inner == 1).sum(), 1)
    model = Pipeline([("pre", make_preprocessor()),
                      ("model", XGBClassifier(**params, scale_pos_weight=scale_pos_weight,
                                              eval_metric="auc", tree_method=XGB_TREE_METHOD,
                                              device=XGB_DEVICE, n_jobs=-1, random_state=RANDOM_STATE))])
    return _fit_and_score(model, X_tr_inner, y_tr_inner, X_val_inner, y_val_inner,
                          roc_auc_score, proba=True)


def objective_lgbm(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 150, 500),
        "num_leaves": trial.suggest_int("num_leaves", 15, 127),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2, log=True),
        "feature_fraction": trial.suggest_float("feature_fraction", 0.6, 1.0),
        "bagging_fraction": trial.suggest_float("bagging_fraction", 0.6, 1.0),
        "min_child_samples": trial.suggest_int("min_child_samples", 10, 100),
    }
    model = Pipeline([("pre", make_preprocessor()),
                      ("model", LGBMClassifier(**params, class_weight="balanced",
                                               n_jobs=-1, random_state=RANDOM_STATE, verbosity=-1))])
    return _fit_and_score(model, X_tr_inner, y_tr_inner, X_val_inner, y_val_inner,
                          roc_auc_score, proba=True)


def _run_study(study_name, objective, target_trials, direction):
    """Create-or-load a persistent Optuna study on OPTUNA_STORAGE and top it up
    to `target_trials` completed trials. Every trial is written to a SQLite file
    next to the checkpoints, so an interrupted search resumes trial-by-trial."""
    study = optuna.create_study(
        study_name=study_name, storage=OPTUNA_STORAGE, load_if_exists=True,
        direction=direction, sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE),
    )
    done = sum(t.state == optuna.trial.TrialState.COMPLETE for t in study.trials)
    remaining = max(0, target_trials - done)
    if remaining:
        print(f"  {study_name}: {done}/{target_trials} trials stored -- running {remaining} more")
        study.optimize(objective, n_trials=remaining, show_progress_bar=True,
                       callbacks=[_gc_after_trial])
    else:
        print(f"  {study_name}: all {target_trials} trials already stored -- skipping")
    return study


t0 = time.time()
study_xgb = _run_study("xgb_classifier_hpo", objective_xgb, OPTUNA_TRIALS_CLASSIFIER, "maximize")
print(f"XGBoost HPO: best val AUC={study_xgb.best_value:.4f} in {time.time() - t0:.0f}s")
print("Best params:", study_xgb.best_params)
```

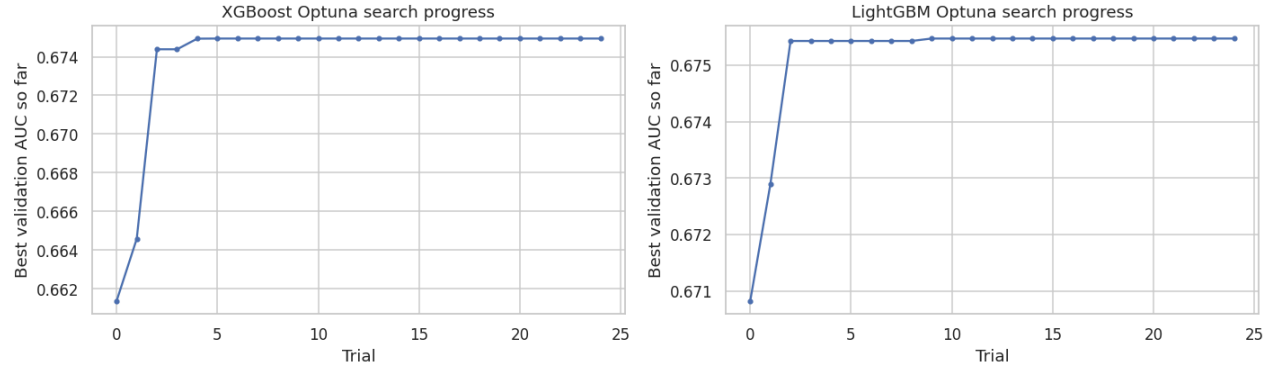
```
  xgb_classifier_hpo: all 25 trials already stored -- skipping
XGBoost HPO: best val AUC=0.6749 in 2s
Best params: {'n_estimators': 433, 'max_depth': 6, 'learning_rate': 0.013399060561509796, 'subsample': 0.87369321
```

```
t0 = time.time()
study_lgbm = _run_study("lgbm_classifier_hpo", objective_lgbm, OPTUNA_TRIALS_CLASSIFIER, "maximize")
print(f"LightGBM HPO: best val AUC={study_lgbm.best_value:.4f} in {time.time() - t0:.0f}s")
print("Best params:", study_lgbm.best_params)
```

```
  lgbm_classifier_hpo: all 25 trials already stored -- skipping
LightGBM HPO: best val AUC=0.6755 in 0s
Best params: {'n_estimators': 359, 'num_leaves': 119, 'learning_rate': 0.01303561122512888, 'feature_fraction': 0
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
for ax, study, name in zip(axes, [study_xgb, study_lgbm], ["XGBoost", "LightGBM"]):
    trials = study.trials_dataframe()
    ax.plot(trials["number"], trials["value"].cummax(), marker="o", markersize=3)
    ax.set_xlabel("Trial")
    ax.set_ylabel("Best validation AUC so far")
    ax.set_title(f"{name} Optuna search progress")
save_fig(fig, "13_optuna_search_progress")
```

```
[saved] 13_optuna_search_progress.png
```



8.1 Refit tuned models on the full production training set, evaluate on 2023

```
scale_pos_weight_full = (y_train_prod == 0).sum() / max((y_train_prod == 1).sum(), 1)


def _build_final(name):
    if name == "xgboost_tuned":
        return Pipeline([("pre", make_preprocessor()),
                         ("model", XGBClassifier(**study_xgb.best_params, scale_pos_weight=scale_pos_weight_ful
                                                 eval_metric="auc", tree_method=XGB_TREE_METHOD,
                                                 device=XGB_DEVICE, n_jobs=-1, random_state=RANDOM_STATE))])
    if name == "lightgbm_tuned":
        return Pipeline([("pre", make_preprocessor()),
                         ("model", LGBMClassifier(**study_lgbm.best_params, class_weight="balanced",
                                                  n_jobs=-1, random_state=RANDOM_STATE, verbosity=-1))])
    if name in ("logistic_regression", "random_forest"):
        return build_model(name)
    raise ValueError(name)


# One checkpoint per final model: a resume reloads whichever fits are done and
# only re-fits the rest. CHECKPOINT_MODELS = False (setup cell) skips persisting
# the fitted objects.
final_models = {}
for name in ["logistic_regression", "random_forest", "xgboost_tuned", "lightgbm_tuned"]:
    ck = f"final_{name}"
    if ckpt_exists(ck):
        final_models[name] = ckpt_load(ck)
        continue
    t0 = time.time()
```

```
        model = _build_final(name)
        model.fit(X_train_prod, y_train_prod)
        metrics, proba, _ = evaluate_classifier(model, X_test_prod, y_test_prod)
        final_models[name] = (model, metrics, proba)
        if CHECKPOINT_MODELS:
            ckpt_save(ck, final_models[name])
        print(f"  fitted {name} in {time.time() - t0:.0f}s")
        gc.collect()

    # Named handles used further down (SHAP picks the best; the save-all cell dumps each).
    final_logreg = final_models["logistic_regression"][0]
    final_rf = final_models["random_forest"][0]
    final_xgb = final_models["xgboost_tuned"][0]
    final_lgbm = final_models["lightgbm_tuned"][0]

    for name, (_, m, _) in final_models.items():
        print(f"[{name:20s}] AUC={m['auc_roc']:.4f} F1={m['f1']:.4f} Precision={m['precision']:.4f} "
              f"Recall={m['recall']:.4f} Brier={m['brier']:.4f}")

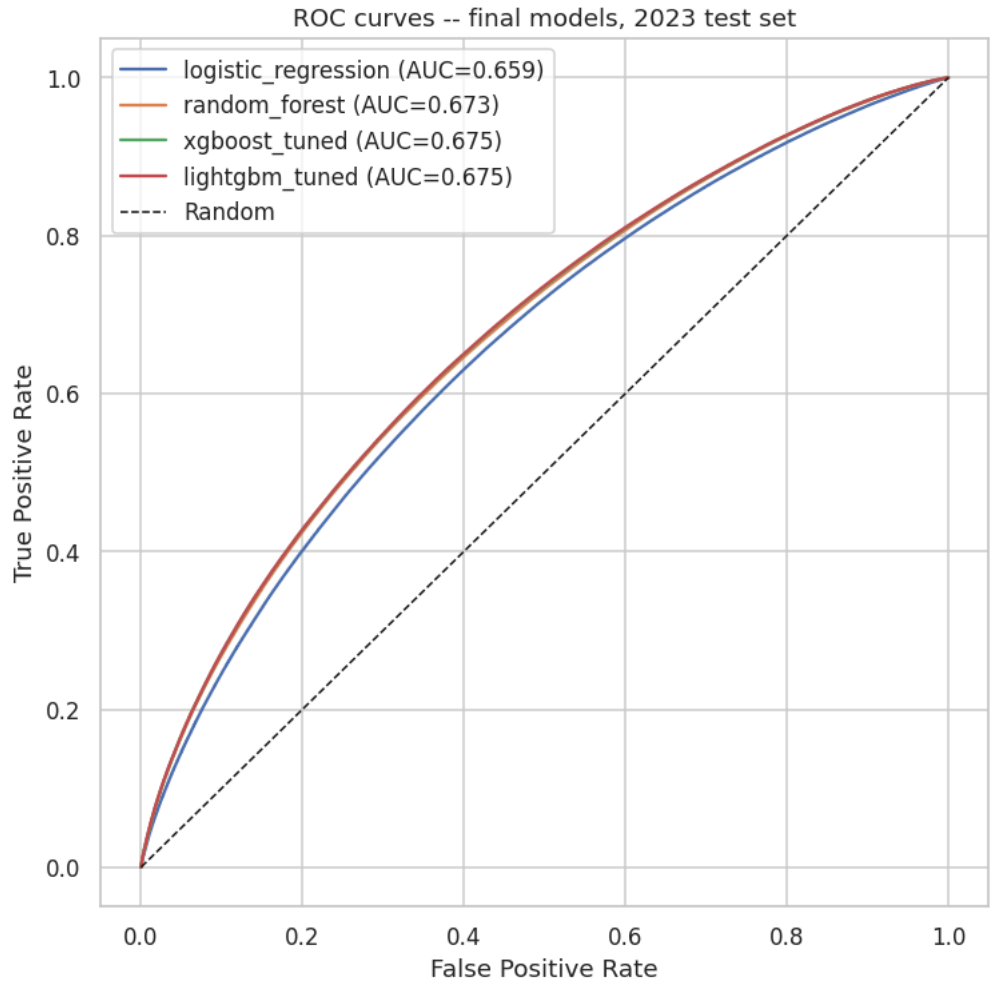
    pd.DataFrame({name: m for name, (_, m, _) in final_models.items()}).T.to_csv(
        OUTPUT_DIR / "final_tuned_model_metrics.csv"
    )
```

```
  [checkpoint restored] final_logistic_regression  (skipping recompute)
  [checkpoint restored] final_random_forest  (skipping recompute)
  [checkpoint restored] final_xgboost_tuned  (skipping recompute)
  [checkpoint restored] final_lightgbm_tuned  (skipping recompute)
[logistic_regression ] AUC=0.6594 F1=0.3971 Precision=0.2884 Recall=0.6372 Brier=0.2367
[random_forest       ] AUC=0.6730 F1=0.4052 Precision=0.2959 Recall=0.6423 Brier=0.2317
[xgboost_tuned       ] AUC=0.6750 F1=0.4066 Precision=0.2962 Recall=0.6482 Brier=0.2321
[lightgbm_tuned      ] AUC=0.6755 F1=0.4072 Precision=0.2974 Recall=0.6455 Brier=0.2316
```

8.2 ROC curves, confusion matrices, calibration -- final tuned models on 2023 test set

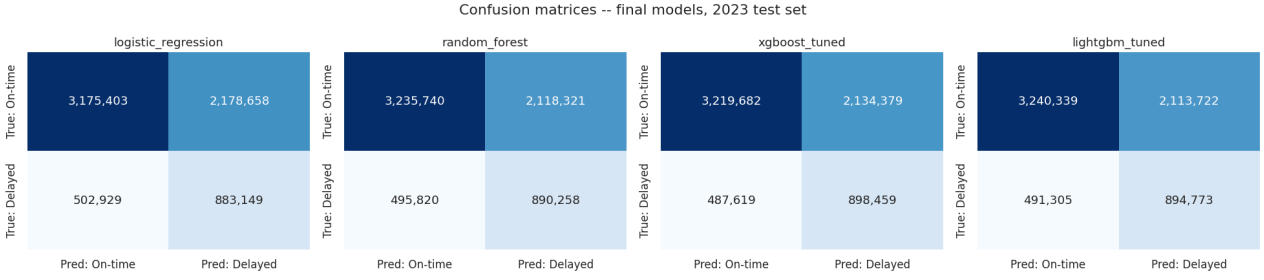
```
fig, ax = plt.subplots(figsize=(7, 7))
for name, (_, m, proba) in final_models.items():
    fpr, tpr, _ = roc_curve(y_test_prod, proba)
    ax.plot(fpr, tpr, label=f"{name} (AUC={m['auc_roc']:.3f})")
ax.plot([0, 1], [0, 1], "k--", linewidth=1, label="Random")
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
ax.set_title("ROC curves -- final models, 2023 test set")
ax.legend()
save_fig(fig, "14_roc_curves")
```

```
[saved] 14_roc_curves.png
```




```
fig, axes = plt.subplots(1, 4, figsize=(18, 4))
for ax, (name, (_, m, proba)) in zip(axes, final_models.items()):
    pred = (proba >= 0.5).astype(int)
    cm = confusion_matrix(y_test_prod, pred, labels=[0, 1])
    sns.heatmap(cm, annot=True, fmt=",d", cmap="Blues", ax=ax, cbar=False,
                xticklabels=["Pred: On-time", "Pred: Delayed"],
                yticklabels=["True: On-time", "True: Delayed"])
    ax.set_title(name)
fig.suptitle("Confusion matrices -- final models, 2023 test set")
save_fig(fig, "15_confusion_matrices")
```

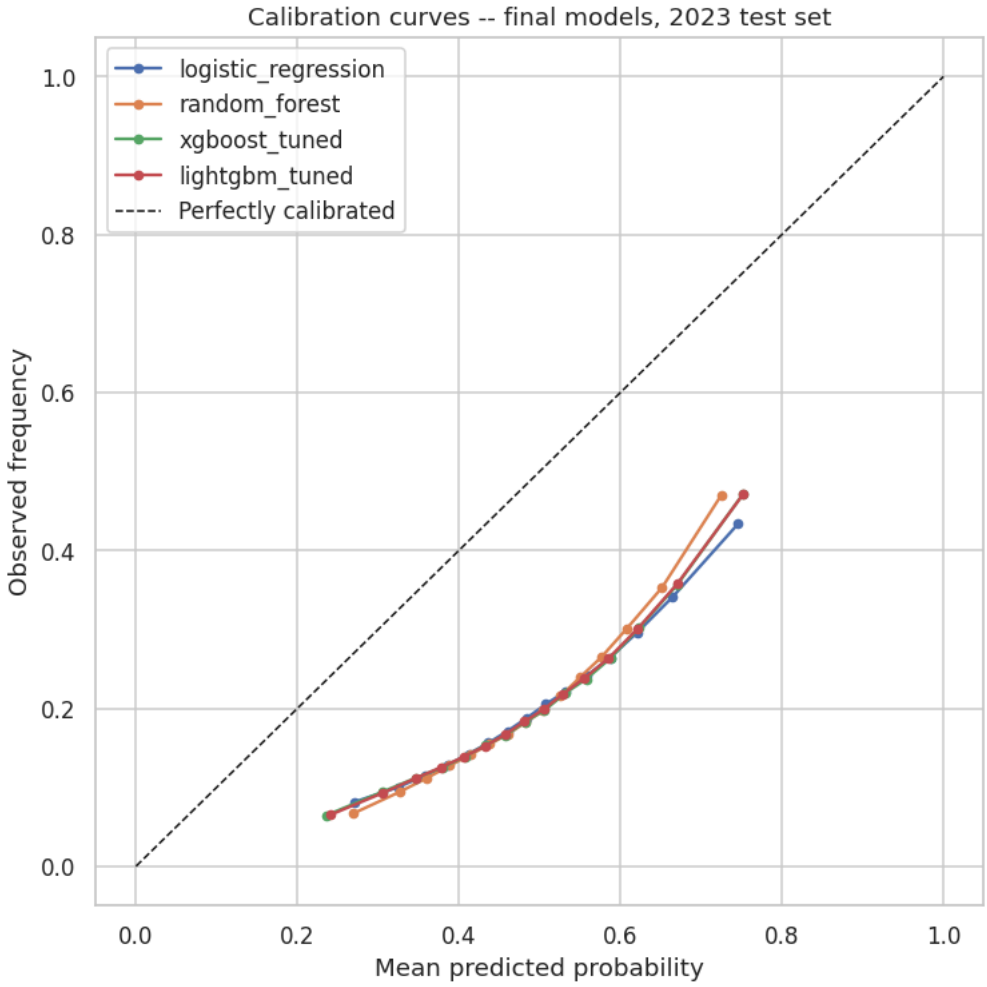
```
[saved] 15_confusion_matrices.png
```




```
fig, ax = plt.subplots(figsize=(7, 7))
for name, (_, m, proba) in final_models.items():
    frac_pos, mean_pred = calibration_curve(y_test_prod, proba, n_bins=15, strategy="quantile")
    ax.plot(mean_pred, frac_pos, marker="o", markersize=4, label=name)
ax.plot([0, 1], [0, 1], "k--", linewidth=1, label="Perfectly calibrated")
```

```
ax.set_xlabel("Mean predicted probability")
ax.set_ylabel("Observed frequency")
ax.set_title("Calibration curves -- final models, 2023 test set")
ax.legend()
save_fig(fig, "16_calibration_curves")
```

```
[saved] 16_calibration_curves.png
```



9. Model Interpretability (SHAP)

```
if ckpt_exists("shap"):
    _s = ckpt_load("shap")
    best_model_name = _s["best_model_name"]
    X_shap, shap_values, feature_names = _s["X_shap"], _s["shap_values"], _s["feature_names"]
    best_model = final_models[best_model_name][0]
    print(f"Best model by test AUC: {best_model_name}  (SHAP restored for {len(X_shap):,} rows)")
else:
    best_model_name = max(final_models, key=lambda k: final_models[k][1]["auc_roc"])
    best_model, _, _ = final_models[best_model_name]
    print(f"Best model by test AUC: {best_model_name}")

    X_test_transformed = best_model.named_steps["pre"].transform(X_test_prod)
    feature_names = best_model.named_steps["pre"].get_feature_names_out()
    X_test_transformed = pd.DataFrame(
        X_test_transformed.toarray() if hasattr(X_test_transformed, "toarray") else X_test_transformed,
        columns=feature_names,
    )

    shap_sample_idx = np.random.RandomState(RANDOM_STATE).choice(
        len(X_test_transformed), size=min(SHAP_SAMPLE_SIZE, len(X_test_transformed)), replace=False
    )
    X_shap = X_test_transformed.iloc[shap_sample_idx].copy()
    del X_test_transformed
    gc.collect()

    model_obj = best_model.named_steps["model"]
    if best_model_name == "logistic_regression":
        explainer = shap.LinearExplainer(model_obj, X_shap)
    else:
```

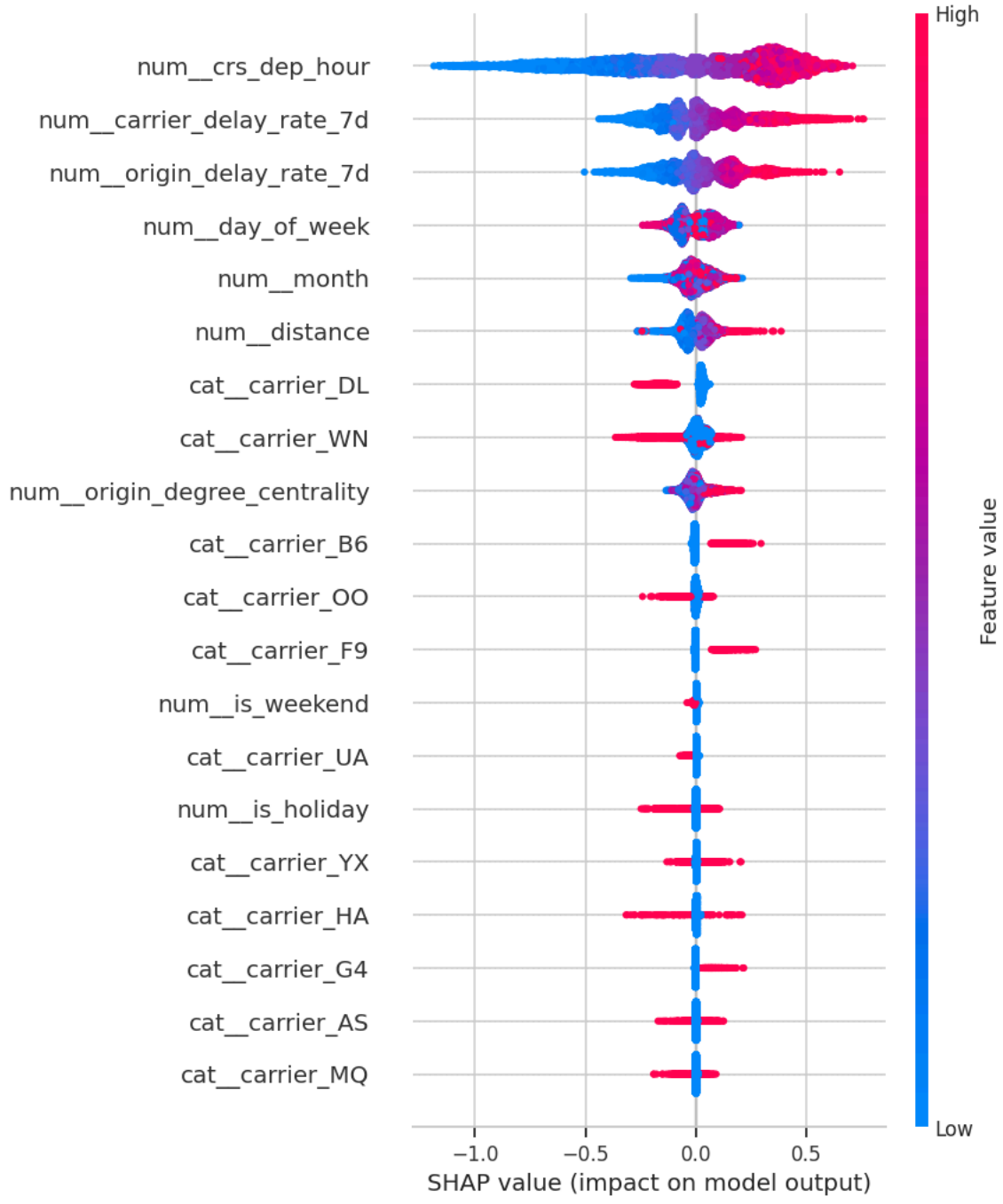
```
    explainer = shap.TreeExplainer(model_obj)  # random_forest, xgboost_tuned, lightgbm_tuned
shap_values = explainer.shap_values(X_shap)
if isinstance(shap_values, list):
    shap_values = shap_values[1]  # positive class for binary classifiers that return a list

ckpt_save("shap", {"best_model_name": best_model_name, "X_shap": X_shap,
                   "shap_values": shap_values, "feature_names": feature_names})
print(f"SHAP values computed on {len(X_shap):,} sampled test rows")
```

```
  [checkpoint restored] shap  (skipping recompute)
Best model by test AUC: lightgbm_tuned  (SHAP restored for 8,000 rows)
```

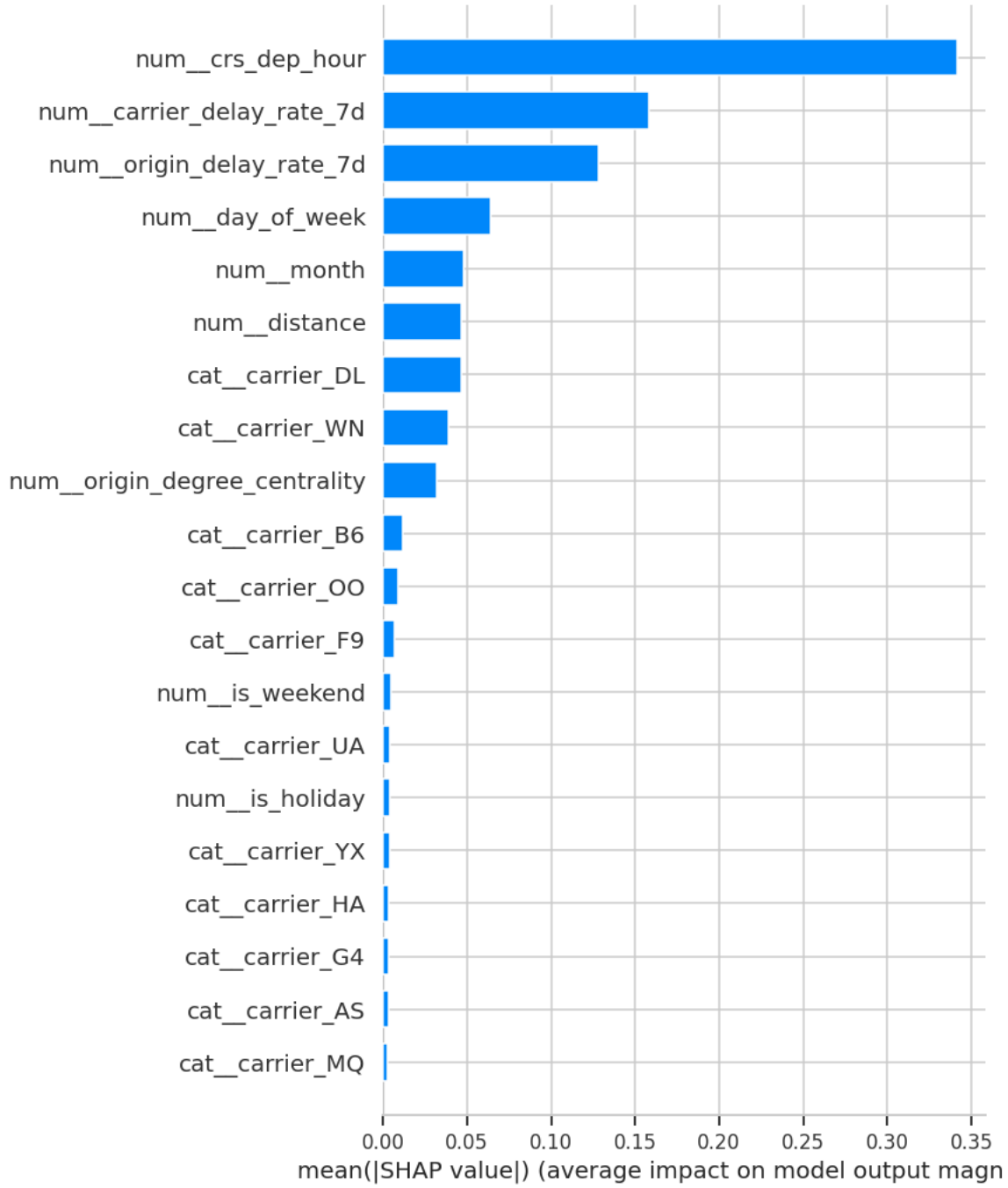
```
# shap.summary_plot creates its own figure; retrieve it with plt.gcf().
shap.summary_plot(shap_values, X_shap, show=False)
save_fig(plt.gcf(), "17_shap_summary_beeswarm")
```

```
[saved] 17_shap_summary_beeswarm.png
```




```
shap.summary_plot(shap_values, X_shap, plot_type="bar", show=False)
save_fig(plt.gcf(), "18_shap_mean_abs_importance")
```

```
[saved] 18_shap_mean_abs_importance.png
```



10. Delay-Duration Regression Sub-Model

```
train_delayed = df[(df["year"].isin(WALK_FORWARD_FOLDS[2]["train_years"])) & (df["arr_del15"] == 1)]
test_delayed = df[(df["year"] == WALK_FORWARD_FOLDS[2]["test_year"]) & (df["arr_del15"] == 1)]
print(f"Delayed flights -- train: {len(train_delayed):,}, test: {len(test_delayed):,}")

reg_val_cutoff = train_delayed["flight_date"].quantile(0.85)
reg_tr_inner = train_delayed[train_delayed["flight_date"] < reg_val_cutoff]
reg_val_inner = train_delayed[train_delayed["flight_date"] >= reg_val_cutoff]


def objective_reg(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 150, 500),
        "num_leaves": trial.suggest_int("num_leaves", 15, 127),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2, log=True),
        "feature_fraction": trial.suggest_float("feature_fraction", 0.6, 1.0),
        "bagging_fraction": trial.suggest_float("bagging_fraction", 0.6, 1.0),
    }
    model = Pipeline([("pre", make_preprocessor()),
                      ("model", LGBMRegressor(**params, n_jobs=-1, random_state=RANDOM_STATE, verbosity=-1))])
    return _fit_and_score(model, reg_tr_inner[ALL_FEATURES], reg_tr_inner["arr_delay_minutes"],
                          reg_val_inner[ALL_FEATURES], reg_val_inner["arr_delay_minutes"],
```

```
                              mean_absolute_error, proba=False)


t0 = time.time()
study_reg = _run_study("lgbm_duration_regressor_hpo", objective_reg, OPTUNA_TRIALS_REGRESSOR, "minimize")
print(f"Duration regressor HPO: best val MAE={study_reg.best_value:.2f} min in {time.time() - t0:.0f}s")
print("Best params:", study_reg.best_params)
```

```
Delayed flights -- train: 4,117,160, test: 1,386,078
  lgbm_duration_regressor_hpo: all 20 trials already stored -- skipping
Duration regressor HPO: best val MAE=47.32 min in 0s
Best params: {'n_estimators': 316, 'num_leaves': 102, 'learning_rate': 0.06900761731140262, 'feature_fraction': 0
```

```
if ckpt_exists("final_regressor"):
    _r = ckpt_load("final_regressor")
    final_regressor, reg_metrics = _r["model"], _r["metrics"]
    reg_pred = final_regressor.predict(test_delayed[ALL_FEATURES])
else:
    final_regressor = Pipeline([("pre", make_preprocessor()),
                                ("model", LGBMRegressor(**study_reg.best_params, n_jobs=-1,
                                                        random_state=RANDOM_STATE, verbosity=-1))])
    final_regressor.fit(train_delayed[ALL_FEATURES], train_delayed["arr_delay_minutes"])
    reg_pred = final_regressor.predict(test_delayed[ALL_FEATURES])

    reg_metrics = {
        "mae": mean_absolute_error(test_delayed["arr_delay_minutes"], reg_pred),
        "rmse": float(np.sqrt(mean_squared_error(test_delayed["arr_delay_minutes"], reg_pred))),
        "r2": r2_score(test_delayed["arr_delay_minutes"], reg_pred),
    }
    if CHECKPOINT_MODELS:
        ckpt_save("final_regressor", {"model": final_regressor, "metrics": reg_metrics})

print(f"Duration regressor -- test set (2023, delayed flights only): {reg_metrics}")
pd.Series(reg_metrics).to_csv(OUTPUT_DIR / "duration_regressor_metrics.csv")
```

```
  [checkpoint restored] final_regressor  (skipping recompute)
Duration regressor -- test set (2023, delayed flights only): {'mae': 49.251376560193656, 'rmse': 100.929396321028
```

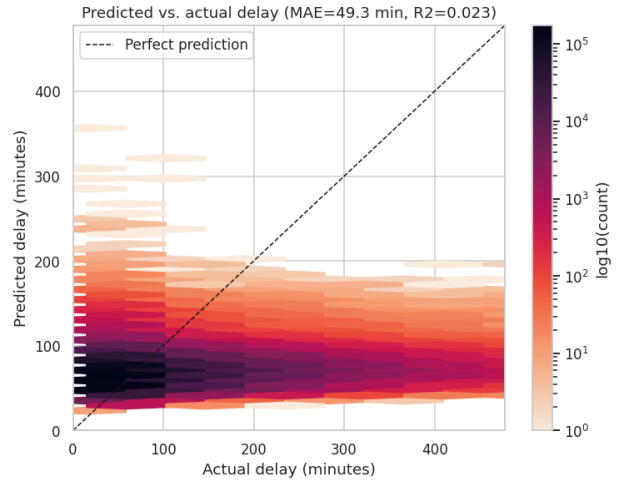
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))

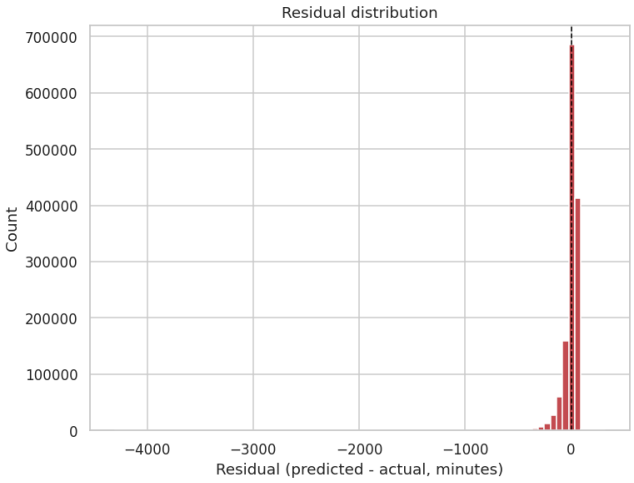
hb = axes[0].hexbin(test_delayed["arr_delay_minutes"], reg_pred, gridsize=50, cmap="rocket_r",
                    mincnt=1, bins="log")
lims = [0, max(test_delayed["arr_delay_minutes"].quantile(0.99), reg_pred.max())]
axes[0].plot(lims, lims, "k--", linewidth=1, label="Perfect prediction")
axes[0].set_xlim(lims); axes[0].set_ylim(lims)
axes[0].set_xlabel("Actual delay (minutes)")
axes[0].set_ylabel("Predicted delay (minutes)")
axes[0].set_title(f"Predicted vs. actual delay (MAE={reg_metrics['mae']:.1f} min, R2={reg_metrics['r2']:.3f})")
axes[0].legend()
fig.colorbar(hb, ax=axes[0], label="log10(count)")

residuals = reg_pred - test_delayed["arr_delay_minutes"].values
axes[1].hist(residuals, bins=80, color="#C44E52")
axes[1].axvline(0, color="black", linestyle="--", linewidth=1)
axes[1].set_xlabel("Residual (predicted - actual, minutes)")
axes[1].set_ylabel("Count")
axes[1].set_title("Residual distribution")

save_fig(fig, "19_duration_regressor_predicted_vs_actual")
```

```
[saved] 19_duration_regressor_predicted_vs_actual.png
```





11. Fairness & Bias Evaluation

```
def geo_quadrant(lat, lon):
    """Vectorised NS/EW quadrant from origin coordinates (no external region data --
    ~39N and ~95W are simple splits of the continental US, not an official boundary)."""
    ns = np.where(lat >= 39, "N", "S")
    ew = np.where(lon <= -95, "W", "E")
    return pd.Series(ns, index=lat.index) + pd.Series(ew, index=lat.index)


test_prod_full = df[df["year"] == WALK_FORWARD_FOLDS[2]["test_year"]].copy()
test_prod_full["geo_quadrant"] = geo_quadrant(test_prod_full["origin_lat"], test_prod_full["origin_lon"])
# Re-predict directly on test_prod_full rather than reusing an earlier proba array,
# so there is no dependence on row-order alignment between DataFrame slices.
test_prod_full["pred_proba"] = best_model.predict_proba(test_prod_full[ALL_FEATURES])[:, 1]
test_prod_full["pred"] = (test_prod_full["pred_proba"] >= 0.5).astype(int)


def fairness_table(group_col):
    rows = []
    for g, gdf in test_prod_full.groupby(group_col, observed=True):
        if len(gdf) < 200:
            continue
        tn, fp, fn, tp = confusion_matrix(gdf["arr_del15"], gdf["pred"], labels=[0, 1]).ravel()
        tpr = tp / (tp + fn) if (tp + fn) else np.nan
        fpr = fp / (fp + tn) if (fp + tn) else np.nan
        rows.append({
            "group": g, "n": len(gdf),
            "auc": roc_auc_score(gdf["arr_del15"], gdf["pred_proba"]) if gdf["arr_del15"].nunique() > 1 else np.
            "f1": f1_score(gdf["arr_del15"], gdf["pred"]),
            "tpr": tpr, "fpr": fpr,
            "positive_pred_rate": gdf["pred"].mean(),
        })
    t = pd.DataFrame(rows).sort_values("n", ascending=False)
    eq_odds_gap = max(t["tpr"].max() - t["tpr"].min(), t["fpr"].max() - t["fpr"].min())
    dp_gap = t["positive_pred_rate"].max() - t["positive_pred_rate"].min()
    return t, eq_odds_gap, dp_gap


if ckpt_exists("fairness"):
    fairness_results = ckpt_load("fairness")
else:
    fairness_results = {}
    for group_col, label in [("carrier", "Carrier"), ("origin_hub_tier", "Origin hub tier"), ("geo_quadrant", "G
        table, eo_gap, dp_gap = fairness_table(group_col)
        fairness_results[label] = {"table": table, "equalised_odds_gap": eo_gap,
                                   "demographic_parity_gap": dp_gap, "group_col": group_col}
```

```
    ckpt_save("fairness", fairness_results)

for label, res in fairness_results.items():
    print(f"\n--- {label} ---  equalised-odds gap={res['equalised_odds_gap']:.3f}  "
          f"demographic-parity gap={res['demographic_parity_gap']:.3f}")
    print(res["table"].to_string(index=False))
    res["table"].to_csv(OUTPUT_DIR / f"fairness_{res['group_col']}.csv", index=False)
```

```
  [checkpoint restored] fairness  (skipping recompute)

--- Carrier ---  equalised-odds gap=0.577  demographic-parity gap=0.585
group       n      auc       f1      tpr      fpr  positive_pred_rate
   WN 1421238 0.709809 0.448647 0.725909 0.421201            0.487656
   DL  972931 0.655782 0.326780 0.395217 0.200353            0.232250
   AA  928058 0.651828 0.419731 0.704765 0.495069            0.543388
   UA  720032 0.661732 0.397137 0.656551 0.428269            0.475314
   OO  661596 0.600809 0.288537 0.408052 0.275501            0.297035
   YX  286490 0.622306 0.271412 0.526993 0.354179            0.376760
   B6  267915 0.675938 0.517140 0.882573 0.693684            0.752589
   NK  258838 0.665636 0.494232 0.795474 0.585442            0.646648
   AS  242643 0.611261 0.350398 0.572364 0.414077            0.445160
   MQ  224695 0.614022 0.332880 0.530704 0.367745            0.397330
   9E  196905 0.612689 0.271919 0.410831 0.260428            0.281359
   OH  191072 0.619620 0.318649 0.488693 0.314583            0.343514
   F9  173459 0.657459 0.502124 0.903934 0.777759            0.817421
   G4  114425 0.658483 0.457541 0.794759 0.587411            0.641145
   HA   79842 0.706727 0.466596 0.676021 0.380396            0.450590

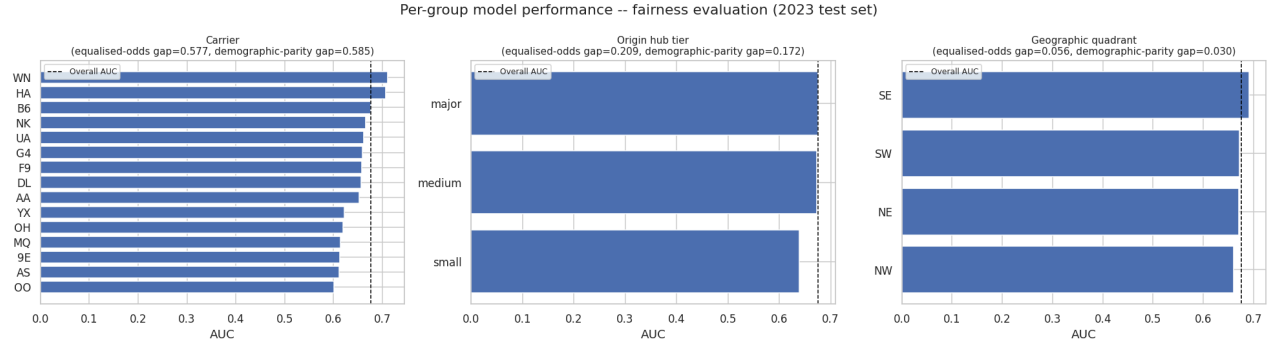
--- Origin hub tier ---  equalised-odds gap=0.209  demographic-parity gap=0.172
 group       n      auc       f1      tpr      fpr  positive_pred_rate
 major 4848457 0.675429 0.418083 0.678017 0.430200            0.483622
medium 1656819 0.672296 0.374841 0.556432 0.312211            0.356422
 small  234863 0.638870 0.337081 0.469454 0.278072            0.311458

--- Geographic quadrant ---  equalised-odds gap=0.056  demographic-parity gap=0.030
group       n      auc       f1      tpr      fpr  positive_pred_rate
   SW 2051483 0.671101 0.401476 0.650246 0.405518            0.455274
   SE 1954703 0.690408 0.427127 0.663133 0.389097            0.447329
   NE 1887882 0.670171 0.398399 0.638844 0.396343            0.445267
   NW  846071 0.660535 0.393782 0.606662 0.378279            0.424906
```

```
overall_auc = roc_auc_score(test_prod_full["arr_del15"], test_prod_full["pred_proba"])

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, (label, res) in zip(axes, fairness_results.items()):
    t = res["table"].sort_values("auc")
    ax.barh(t["group"].astype(str), t["auc"], color="#4C72B0")
    ax.axvline(overall_auc, color="black", linestyle="--", linewidth=1, label="Overall AUC")
    ax.set_xlabel("AUC")
    ax.set_title(f"{label}\n(equalised-odds gap={res['equalised_odds_gap']:.3f}, "
                 f"demographic-parity gap={res['demographic_parity_gap']:.3f})", fontsize=10)
    ax.legend(fontsize=8)
fig.suptitle("Per-group model performance -- fairness evaluation (2023 test set)")
save_fig(fig, "20_fairness_group_auc")
```

```
[saved] 20_fairness_group_auc.png
```



12. COVID-19 Period Distribution-Shift Check

```
if ckpt_exists("covid_check"):
    covid_check_df = ckpt_load("covid_check")
else:
    covid_check_rows = []
    for year in [2020, 2021, WALK_FORWARD_FOLDS[2]["test_year"]]:
        year_df = df[df["year"] == year]
        proba = final_models[best_model_name][0].predict_proba(year_df[ALL_FEATURES])[:, 1]
        pred = (proba >= 0.5).astype(int)
        covid_check_rows.append({
            "year": year,
            "label": "In-distribution (2023)" if year == WALK_FORWARD_FOLDS[2]["test_year"] else "COVID period (
            "n": len(year_df),
            "actual_delay_rate": year_df["arr_del15"].mean(),
            "auc": roc_auc_score(year_df["arr_del15"], proba),
            "f1": f1_score(year_df["arr_del15"], pred),
        })
    covid_check_df = pd.DataFrame(covid_check_rows)
    ckpt_save("covid_check", covid_check_df)

covid_check_df.to_csv(OUTPUT_DIR / "covid_distribution_shift_check.csv", index=False)
covid_check_df
```

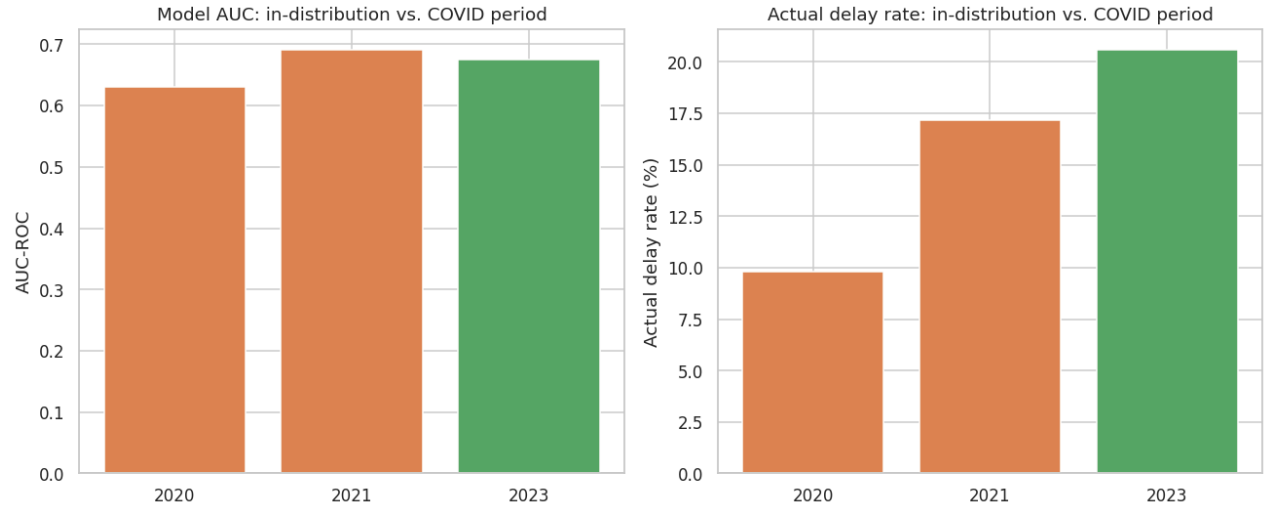
```
[checkpoint restored] covid_check  (skipping recompute)
```

	year	label	n	actual_delay_rate	auc	f1
0	2020	COVID period (excluded from training)	4396951	0.098168	0.630846	0.198456
1	2021	COVID period (excluded from training)	5874238	0.171867	0.690233	0.381987
2	2023	In-distribution (2023)	6740139	0.205645	0.675471	0.407217

Next steps: New interactive sheet

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
colors = ["#DD8452" if "COVID" in l else "#55A868" for l in covid_check_df["label"]]
axes[0].bar(covid_check_df["year"].astype(str), covid_check_df["auc"], color=colors)
axes[0].set_ylabel("AUC-ROC")
axes[0].set_title("Model AUC: in-distribution vs. COVID period")
axes[1].bar(covid_check_df["year"].astype(str), covid_check_df["actual_delay_rate"]*100, color=colors)
axes[1].set_ylabel("Actual delay rate (%)")
axes[1].set_title("Actual delay rate: in-distribution vs. COVID period")
save_fig(fig, "21_covid_distribution_shift")
```

```
[saved] 21_covid_distribution_shift.png
```



13. Model Comparison Summary

```
summary_rows = []
for name, (_, m, _) in final_models.items():
    summary_rows.append({"model": name, **m})
summary_rows.append({"model": "duration_regressor (LightGBM, MAE/RMSE shown as auc_roc/f1 cols)",
                     "auc_roc": reg_metrics["mae"], "f1": reg_metrics["rmse"],
                     "precision": reg_metrics["r2"], "recall": np.nan, "brier": np.nan})
final_summary_df = pd.DataFrame(summary_rows)
final_summary_df.to_csv(OUTPUT_DIR / "final_model_comparison_summary.csv", index=False)
final_summary_df
```

	model	auc_roc	f1	precision	recall	brier
0	logistic_regression	0.659378	0.397110	0.288440	0.637157	0.236690
1	random_forest	0.672976	0.405155	0.295906	0.642286	0.231686
2	xgboost_tuned	0.674978	0.406642	0.296244	0.648202	0.232134
3	lightgbm_tuned	0.675471	0.407217	0.297415	0.645543	0.231571
4	duration_regressor (LightGBM, MAE/RMSE shown a...	49.251377	100.929396	0.022981	NaN	NaN

Next steps: New interactive sheet

14. Save All Figures & Models, Download as ZIP

```
joblib.dump(final_logreg, MODELS_DIR / "logistic_regression_final.pkl")
joblib.dump(final_rf, MODELS_DIR / "random_forest_final.pkl")
joblib.dump(final_xgb, MODELS_DIR / "xgboost_tuned_final.pkl")
joblib.dump(final_lgbm, MODELS_DIR / "lightgbm_tuned_final.pkl")
joblib.dump(final_regressor, MODELS_DIR / "lightgbm_duration_regressor_final.pkl")

best_params = {
    "xgboost": study_xgb.best_params,
    "lightgbm_classifier": study_lgbm.best_params,
    "lightgbm_regressor": study_reg.best_params,
}
with open(MODELS_DIR / "best_hyperparameters.json", "w") as f:
    json.dump(best_params, f, indent=2)

with open(OUTPUT_DIR / "run_config.json", "w") as f:
    json.dump({
        "rf_n_estimators": RF_N_ESTIMATORS,
        "xgb_lgbm_n_estimators": XGB_LGBM_N_ESTIMATORS,
        "optuna_trials_classifier": OPTUNA_TRIALS_CLASSIFIER,
        "optuna_trials_regressor": OPTUNA_TRIALS_REGRESSOR,
        "shap_sample_size": SHAP_SAMPLE_SIZE,
        "best_model_by_auc": best_model_name,
        "n_rows_total": int(len(df)),
    }, f, indent=2)

print("Saved models:")
for p in sorted(MODELS_DIR.glob("*")):
    print(" ", p.name)
print("\nSaved figures:")
for p in sorted(FIGURES_DIR.glob("*")):
    print(" ", p.name)
```

```
Saved models:
  best_hyperparameters.json
  lightgbm_duration_regressor_final.pkl
  lightgbm_tuned_final.pkl
  logistic_regression_final.pkl
  random_forest_final.pkl
  xgboost_tuned_final.pkl

Saved figures:
  01_class_balance.png
  02_monthly_delay_trend.png
  03_hour_dow_heatmap.png
  04_delay_rate_by_carrier.png
  05_top_airports_volume_delay.png
  06_delay_duration_violin_by_carrier.png
  07_distance_vs_delay_hexbin.png
  08_correlation_heatmap.png
  09_interactive_monthly_delay_by_carrier.html
  10_interactive_us_airport_map.html
  11_interactive_sunburst_carrier_hub.html
  12_walk_forward_model_comparison.png
  13_optuna_search_progress.png
```

```
  14_roc_curves.png
  15_confusion_matrices.png
  16_calibration_curves.png
  17_shap_summary_beeswarm.png
  18_shap_mean_abs_importance.png
  19_duration_regressor_predicted_vs_actual.png
  20_fairness_group_auc.png
  21_covid_distribution_shift.png
```

```
zip_path = OUTPUT_DIR.parent / f"{OUTPUT_DIR.name}.zip"
with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as zf:
    for p in OUTPUT_DIR.rglob("*"):
        if p.is_file():
            zf.write(p, p.relative_to(OUTPUT_DIR.parent))

print(f"Zipped everything to {zip_path} ({zip_path.stat().st_size/1e6:.1f} MB)")

if IN_COLAB:
    from google.colab import files
    files.download(str(zip_path))
else:
    print(f"Not running in Colab -- find your outputs at: {zip_path.resolve()}")
```

```
Zipped everything to /content/PRT661_outputs.zip (167.7 MB)
```

15. A4 setup and NOAA GHCN-Daily data

```
# ---- A4 configuration (A2 settings are re-used, not redefined) -----------------
from sklearn.metrics import average_precision_score
from sklearn.isotonic import IsotonicRegression
try:
    display
except NameError:                                          # plain Python (not Jupyter/Colab)
    display = print
A4_TRAIN_YEARS = WALK_FORWARD_FOLDS[2]["train_years"]       # 2018, 2019, 2022  (COVID years excluded)
A4_TEST_YEAR = WALK_FORWARD_FOLDS[2]["test_year"]           # 2023
A4_WINDOW_DAYS = 7                                         # calendar days, strictly before the flight date
A4_STATION_RADIUS_KM = 50                                  # airport -> weather-station search radius
A4_STATIONS_PER_ELEMENT = 3                                # fallback stations per airport and element
A4_VAL_FRACTION = 0.15                                     # last 15% of training dates = calibration / thresho
A4_QUANTILES = (0.1, 0.5, 0.9)                             # duration interval (80% central interval)
A4_COST_FN, A4_COST_FP = 3.0, 1.0                          # ops cost of a missed delay vs a false alarm (edital
A4_ALERT_BUDGETS = (0.05, 0.10, 0.20)                      # share of flights an ops team could act on

if IN_COLAB:
    NOAA_DIR = DRIVE_BASE / "noaa"                         # created if missing; files are downloaded if absent
else:
    NOAA_DIR = Path(os.environ.get("PRT661_NOAA_DIR", "./noaa"))
NOAA_YEARS = list(range(2018, 2024))
A4_DIR = OUTPUT_DIR / "a4"
A4_DIR.mkdir(parents=True, exist_ok=True)

# Hyper-parameters tuned in A2 (Optuna, 25 / 20 trials). Used unchanged so the comparison with A2 is like-for-like
# (A2 tuned `bagging_fraction` without `bagging_freq`, so it has no effect in LightGBM; kept as-is for comparabil
_A2_CLF_FALLBACK = dict(n_estimators=359, num_leaves=119, learning_rate=0.013035611225128, feature_fraction=0.678
                        bagging_fraction=0.6180909155642152, min_child_samples=39)
_A2_REG_FALLBACK = dict(n_estimators=316, num_leaves=102, learning_rate=0.06900761731140262, feature_fraction=0.6
                        bagging_fraction=0.7476954781156898)
A4_CLF_PARAMS = dict(study_lgbm.best_params) if "study_lgbm" in globals() else dict(_A2_CLF_FALLBACK)
A4_REG_PARAMS = dict(study_reg.best_params) if "study_reg" in globals() else dict(_A2_REG_FALLBACK)
print("Classifier params:", A4_CLF_PARAMS)
print("Regressor params :", A4_REG_PARAMS)


# ---- NOAA: use the files on Drive; download + filter only what is missing --------
NOAA_ELEMENTS = {"PRCP", "SNOW", "SNWD", "TMAX", "TMIN", "AWND", "WSF2", "WT01", "WT03", "WT04", "WT05", "WT06", "
NOAA_US_PREFIXES = ("US", "RQ", "VQ", "GQ", "AQ")
NOAA_BASE = "https://www.ncei.noaa.gov/pub/data/ghcn/daily"


def ensure_noaa():
    """Same filtering as ingestion/download_noaa.py: US stations, selected elements, quality-flag-passed rows."""
    import gzip
    import requests
    NOAA_DIR.mkdir(parents=True, exist_ok=True)
    for name in ["ghcnd-stations.txt", "ghcnd-inventory.txt"]:
        dest = NOAA_DIR / name
        if dest.exists() and dest.stat().st_size > 0:
            continue
        print(f"[download] {name}")
```

```
        print(f"[download] {name}")
        with requests.get(f"{NOAA_BASE}/{name}", stream=True, timeout=120) as r:
            r.raise_for_status()
            with open(dest, "wb") as f:
                for block in r.iter_content(1 << 20):
                    f.write(block)
    cols = ["station", "date", "element", "value", "m_flag", "q_flag", "s_flag", "obs_time"]
    for year in NOAA_YEARS:
        out = NOAA_DIR / f"noaa_daily_{year}.parquet"
        if out.exists():
            continue
        print(f"[stream] NOAA {year} (about 170 MB compressed; filtered while streaming)")
        parts = []
        with requests.get(f"{NOAA_BASE}/by_year/{year}.csv.gz", stream=True, timeout=300) as r:
            r.raise_for_status()
            r.raw.decode_content = False
            with gzip.GzipFile(fileobj=r.raw) as gz:
                for chunk in pd.read_csv(gz, header=None, names=cols, chunksize=2_000_000, dtype=str, keep_defaul
                    keep = (chunk["station"].str.startswith(NOAA_US_PREFIXES) & chunk["element"].isin(NOAA_ELEMENT
                            & (chunk["q_flag"] == ""))
                    sub = chunk.loc[keep, ["station", "date", "element", "value"]]
                    if len(sub):
                        parts.append(sub.assign(date=pd.to_datetime(sub["date"], format="%Y%m%d"),
                                                value=pd.to_numeric(sub["value"]).astype("float32")))
        res = pd.concat(parts, ignore_index=True)
        res["station"] = res["station"].astype("category")
        res["element"] = res["element"].astype("category")
        res.to_parquet(out.with_suffix(".part"), index=False)
        out.with_suffix(".part").replace(out)
        print(f"[ok] {out.name}: {len(res):,} observations")


ensure_noaa()
for p in sorted(NOAA_DIR.glob("*")):
    print(f"  {p.name:28s} {p.stat().st_size / 1e6:8.1f} MB")
```

```
Classifier params: {'n_estimators': 359, 'num_leaves': 119, 'learning_rate': 0.01303561122512888, 'feature_fracti
Regressor params : {'n_estimators': 316, 'num_leaves': 102, 'learning_rate': 0.06900761731140262, 'feature_fracti
[download] ghcnd-stations.txt
[download] ghcnd-inventory.txt
[stream] NOAA 2018 (about 170 MB compressed; filtered while streaming)
[ok] noaa_daily_2018.parquet: 20,300,850 observations
[stream] NOAA 2019 (about 170 MB compressed; filtered while streaming)
[ok] noaa_daily_2019.parquet: 20,323,395 observations
[stream] NOAA 2020 (about 170 MB compressed; filtered while streaming)
[ok] noaa_daily_2020.parquet: 20,794,678 observations
[stream] NOAA 2021 (about 170 MB compressed; filtered while streaming)
[ok] noaa_daily_2021.parquet: 21,785,679 observations
[stream] NOAA 2022 (about 170 MB compressed; filtered while streaming)
[ok] noaa_daily_2022.parquet: 21,867,810 observations
[stream] NOAA 2023 (about 170 MB compressed; filtered while streaming)
[ok] noaa_daily_2023.parquet: 22,272,111 observations
  ghcnd-inventory.txt              36.4 MB
  ghcnd-stations.txt               11.4 MB
  noaa_daily_2018.parquet          64.2 MB
  noaa_daily_2019.parquet          64.3 MB
  noaa_daily_2020.parquet          65.0 MB
  noaa_daily_2021.parquet          66.9 MB
  noaa_daily_2022.parquet          67.0 MB
  noaa_daily_2023.parquet          68.3 MB
```

16. Audit of the A2 rolling delay-rate features

The A2 report called `carrier_delay_rate_7d` / `origin_delay_rate_7d` "trailing 7-day" features. The SQL window (`ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING`) runs over a **per-day aggregate table**, so it really means *the previous seven days on which that carrier / airport operated*, averaged **without weighting by flight count**. This section measures how far that differs from a true calendar window, then rebuilds the feature as the **flight-weighted delay rate over the seven calendar days strictly before the flight date** (`[d−7, d−1]`). Both versions use past outcomes only; the point here is that the definition is now exact and testable.

```
A4_START = np.datetime64("2018-01-01")
A4_T = int((np.datetime64("2023-12-31") - A4_START).astype(int)) + 1      # 2,191 calendar days


def _as_cat(s):
    return s if isinstance(s.dtype, pd.CategoricalDtype) else s.astype("category")


def _day_index(dates):
    """Calendar-day index (0 = 2018-01-01) of a datetime Series."""
    t = (dates.values.astype("datetime64[D]") - A4_START).astype(np.int64)
```

```
    assert t.min() >= 0 and t.max() < A4_T, "flight_date outside 2018-2023"
    return t


def _daily_counts(frame, key):
    """Dense [key x day] arrays: flights N and delayed flights D."""
    cat = _as_cat(frame[key])
    codes = cat.cat.codes.to_numpy().astype(np.int64)
    assert codes.min() >= 0, f"missing {key}"
    flat = codes * A4_T + _day_index(frame["flight_date"])
    K = len(cat.cat.categories)
    N = np.bincount(flat, minlength=K * A4_T).reshape(K, A4_T)
    D = np.bincount(flat[frame["arr_del15"].to_numpy() == 1], minlength=K * A4_T).reshape(K, A4_T)
    return N, D, list(cat.cat.categories)


def _rolling_rate(N, D, window=A4_WINDOW_DAYS):
    """Flight-weighted delay rate over calendar days [t-window, t-1] (the flight's own day is excluded).
    Returns (rate, n_flights_in_window); rate is NaN when the window holds no flights."""
    z = np.zeros((N.shape[0], 1), dtype=np.int64)
    CN, CD = np.concatenate([z, np.cumsum(N, axis=1)], axis=1), np.concatenate([z, np.cumsum(D, axis=1)], axis=1
    t = np.arange(N.shape[1])
    lo = np.maximum(t - window, 0)
    n, d = CN[:, t] - CN[:, lo], CD[:, t] - CD[:, lo]
    return np.where(n > 0, d / np.maximum(n, 1), np.nan).astype(np.float32), n


t0 = time.time()
CARR_N, CARR_D, CARR_CATS = _daily_counts(df, "carrier")
ORIG_N, ORIG_D, ORIG_CATS = _daily_counts(df, "origin")
CARR_RATE, CARR_NW = _rolling_rate(CARR_N, CARR_D)
ORIG_RATE, ORIG_NW = _rolling_rate(ORIG_N, ORIG_D)
print(f"Dense daily tables built in {time.time() - t0:.0f}s: carriers {CARR_N.shape}, origin airports {ORIG_N.sh

# (a) How far is "7 preceding active days" from "7 calendar days"?
spans, weights = [], []
for k in range(ORIG_N.shape[0]):
    act = np.flatnonzero(ORIG_N[k] > 0)
    if len(act) > 7:
        spans.append(act[7:] - act[:-7])                # calendar days covered by the 7 preceding ROWS
        weights.append(ORIG_N[k, act[7:]])
spans, weights = np.concatenate(spans), np.concatenate(weights)
span_stats = pd.DataFrame({
    "metric": ["share of airport-days whose 7-row window spans more than 7 calendar days",
               "same, weighted by flights", "95th percentile calendar span of the 7-row window (days)",
               "maximum span (days)"],
    "value": [(spans > 7).mean(), (weights[spans > 7].sum() / weights.sum()), np.percentile(spans, 95), spans.ma
})
display(span_stats)

# (b) Numerical difference between the A2 feature and the calendar-exact one (random 1M rows)
rng = np.random.RandomState(RANDOM_STATE)
idx = np.sort(rng.choice(len(df), size=min(1_000_000, len(df)), replace=False))
t_s = _day_index(df["flight_date"].iloc[idx])
o_s = _as_cat(df["origin"]).cat.codes.to_numpy()[idx]
c_s = _as_cat(df["carrier"]).cat.codes.to_numpy()[idx]
cmp_rows = []
for name, a2_col, new in [("origin", "origin_delay_rate_7d", ORIG_RATE[o_s, t_s]),
                          ("carrier", "carrier_delay_rate_7d", CARR_RATE[c_s, t_s])]:
    a2 = df[a2_col].to_numpy()[idx].astype(np.float32)
    ok = ~np.isnan(new)
    cmp_rows.append({"feature": name, "pearson_r": np.corrcoef(a2[ok], new[ok])[0, 1],
                     "mean_abs_diff": np.abs(a2[ok] - new[ok]).mean(),
                     "share_diff_gt_0.05": (np.abs(a2[ok] - new[ok]) > 0.05).mean()})
a2_vs_calendar = pd.DataFrame(cmp_rows)
display(a2_vs_calendar)
a2_vs_calendar.to_csv(A4_DIR / "a2_vs_calendar_rate_comparison.csv", index=False)
span_stats.to_csv(A4_DIR / "a2_window_span_audit.csv", index=False)
del spans, weights
gc.collect()
```

```
Dense daily tables built in 3s: carriers (19, 2191), origin airports (379, 2191)
```

	metric	value
0	share of airport-days whose 7-row window spans...	0.052677
1	same, weighted by flights	0.003076
2	95th percentile calendar span of the 7-row win...	8.000000
3	maximum span (days)	1593.000000

	feature	pearson_r	mean_abs_diff	share_diff_gt_0.05
0	origin	0.994303	0.003471	0.005557
1	carrier	0.997991	0.002252	0.001410

```
4509
```

Next steps: New interactive sheet New interactive sheet

17. Graph features computed from the training window only

In A2, `origin_degree_centrality` and `origin_hub_tier` used the **full six-year table**, including the 2023 test year. Hub volume and route counts of later years therefore informed earlier predictions. Here they are recomputed from the **training years only** (2018, 2019, 2022); airports that never appear in training receive a neutral fallback (centrality 0, tier `small`) and are counted.

```
def graph_features_from_training(train_years):
    m = df["year"].isin(train_years).to_numpy()
    tr = df.loc[m, ["origin", "destination"]]
    degree = tr.groupby("origin", observed=True)["destination"].nunique()
    degree_norm = (degree - degree.min()) / (degree.max() - degree.min())
    volume = tr["origin"].value_counts()
    volume = volume[volume > 0]                                    # drop categories with no training flights
    q90, q60 = volume.quantile(0.90), volume.quantile(0.60)
    tier = volume.apply(lambda v: "major" if v >= q90 else ("medium" if v >= q60 else "small"))
    return degree_norm, tier


A4_DEGREE, A4_TIER = graph_features_from_training(A4_TRAIN_YEARS)

# Compare with the A2 (full-six-year) tiers
a2_tier = df.groupby("origin", observed=True)["origin_hub_tier"].first().astype(str)
cmp_tier = pd.DataFrame({"a2_tier_full_6y": a2_tier, "a4_tier_train_only": A4_TIER}).fillna({"a4_tier_train_only
changed = cmp_tier[cmp_tier["a2_tier_full_6y"] != cmp_tier["a4_tier_train_only"]]
test_flights_by_origin = df.loc[df["year"] == A4_TEST_YEAR, "origin"].value_counts()
n_unseen = int((~a2_tier.index.isin(A4_TIER.index)).sum())
print(f"{len(a2_tier)} origin airports | {len(changed)} change tier when computed from training years only | "
      f"{n_unseen} never appear as an origin in the training years (fallback: small / centrality 0)")
print(f"Share of {A4_TEST_YEAR} flights departing from an airport whose tier changed: "
      f"{test_flights_by_origin.reindex(changed.index).fillna(0).sum() / test_flights_by_origin.sum():.2%}")
display(pd.crosstab(cmp_tier["a2_tier_full_6y"], cmp_tier["a4_tier_train_only"]))
cmp_tier.to_csv(A4_DIR / "hub_tier_a2_vs_train_only.csv")
```

```
379 origin airports | 13 change tier when computed from training years only | 2 never appear as an origin in the
Share of 2023 flights departing from an airport whose tier changed: 2.01%
```

a4_tier_train_only	major	medium	small	small (unseen)
a2_tier_full_6y				
major	37	1	0	0
medium	1	108	5	0
small	0	4	221	2

18. NOAA weather features and their availability

Availability rule. GHCN-Daily values are *end-of-day summaries*: the weather of day *d* is only complete after the flights of day *d* have already departed. Using same-day observations as predictors would therefore leak information the dashboard cannot have at prediction time. The deployable weather features use **the previous calendar day (d-1)** at the origin and destination airports (plus a 3-day precipitation total ending d-1). As a clearly-labelled **upper bound, not deployable**, section 20 also trains a variant that uses same-day observations; in production this slot would be filled by a weather *forecast* (e.g. NWS/TAF), which this project does not have.

Each airport is matched to the nearest GHCN stations that report each element within the search radius (the nearest station that has a value that day is used).

```
WX_ELEMENTS = ["PRCP", "SNOW", "SNWD", "TMAX", "TMIN", "AWND", "WSF2", "WT01", "WT03"]
WX_UNIT_DIV = {"PRCP": 10.0, "TMAX": 10.0, "TMIN": 10.0, "AWND": 10.0, "WSF2": 10.0}   # tenths -> mm, deg C, m/


def _build_weather():
    # Airports that appear as an origin or destination, with coordinates.
    o = df.groupby("origin", observed=True)[["origin_lat", "origin_lon"]].first()
    d = df.groupby("destination", observed=True)[["dest_lat", "dest_lon"]].first()
    o.columns, d.columns = ["lat", "lon"], ["lat", "lon"]
    o, d = o.rename_axis("iata"), d.rename_axis("iata")
    ap = pd.concat([o, d[~d.index.isin(o.index)]]).dropna().reset_index()
    ap["iata"] = ap["iata"].astype(str)

    # Station inventory (fixed-width): id, lat, lon, element, first year, last year
    inv = pd.read_fwf(NOAA_DIR / "ghcnd-inventory.txt", colspecs=[(0, 11), (12, 20), (21, 30), (31, 35), (36, 40
                      names=["station", "lat", "lon", "element", "first", "last"], header=None)
    inv = inv[inv["station"].str.startswith(NOAA_US_PREFIXES) & inv["element"].isin(WX_ELEMENTS)
              & (inv["last"] >= NOAA_YEARS[0]) & (inv["first"] <= NOAA_YEARS[-1])]
    print(f"{len(ap)} airports, {inv['station'].nunique():,} candidate stations, {len(inv):,} station-element ro

    con = duckdb.connect()
    con.execute("PRAGMA threads=2")
    con.register("ap", ap)
    con.register("inv", inv)
    con.execute(f"""
        CREATE TABLE cand AS
        SELECT a.iata, i.element, i.station,
               2*6371*asin(sqrt(pow(sin(radians(i.lat-a.lat)/2),2)
                    + cos(radians(a.lat))*cos(radians(i.lat))*pow(sin(radians(i.lon-a.lon)/2),2))) AS dist_km
        FROM ap a JOIN inv i ON abs(i.lat-a.lat) <= 1.0 AND abs(i.lon-a.lon) <= 1.6
    """)
    con.execute(f"""
        CREATE TABLE smap AS
        SELECT iata, element, station, dist_km, rnk FROM (
            SELECT *, row_number() OVER (PARTITION BY iata, element ORDER BY dist_km) AS rnk
            FROM cand WHERE dist_km <= {A4_STATION_RADIUS_KM}
        ) WHERE rnk <= {A4_STATIONS_PER_ELEMENT}
    """)
    files = ", ".join(repr((NOAA_DIR / f"noaa_daily_{y}.parquet").as_posix()) for y in NOAA_YEARS)
    obs = con.execute(f"""
        SELECT m.iata, CAST(o.date AS DATE) AS date, o.element, arg_min(o.value, m.rnk) AS value
        FROM read_parquet([{files}]) o JOIN smap m ON o.station = m.station AND o.element = m.element
        GROUP BY 1, 2, 3
    """).df()
    station_map = con.execute("""SELECT iata, element, station, round(dist_km, 1) AS dist_km, rnk
                                 FROM smap WHERE rnk = 1 ORDER BY iata, element""").df()
    con.close()

    stations = pd.read_fwf(NOAA_DIR / "ghcnd-stations.txt", colspecs=[(0, 11), (12, 20), (21, 30), (31, 37), (38
                           names=["station", "lat", "lon", "elev", "state", "name"], header=None)
    station_map = station_map.merge(stations[["station", "name", "state"]], on="station", how="left")

    ap_index = {c: i for i, c in enumerate(ap["iata"])}
    W = {el: np.full((len(ap), A4_T), np.nan, dtype=np.float32) for el in WX_ELEMENTS}
    a_idx = obs["iata"].map(ap_index).to_numpy()
    t_idx = _day_index(obs["date"])
    for el in WX_ELEMENTS:
        sel = (obs["element"] == el).to_numpy()
        W[el][a_idx[sel], t_idx[sel]] = obs.loc[sel, "value"].to_numpy() / WX_UNIT_DIV.get(el, 1.0)
    coverage = pd.DataFrame({el: [float(np.isfinite(W[el]).mean())] for el in WX_ELEMENTS}, index=["airport-day
    for el in ("WT01", "WT03"):                                    # weather-type flags are only reported when pr
        W[el] = np.nan_to_num(W[el], nan=0.0)
    return {"airports": list(ap["iata"]), "W": W, "coverage": coverage, "station_map": station_map}


WX = ckpt_step("a4_weather_airport_day", _build_weather)
A4_AP_INDEX = {c: i for i, c in enumerate(WX["airports"])}
display(WX["coverage"].T)
prcp_map = WX["station_map"][WX["station_map"]["element"] == "PRCP"]
print(f"Nearest PRCP station: median {prcp_map['dist_km'].median():.1f} km, 95th pct {prcp_map['dist_km'].quanti
      f"{len(WX['airports']) - len(prcp_map)} airports with no PRCP station within {A4_STATION_RADIUS_KM} km")
WX["station_map"].to_csv(A4_DIR / "airport_weather_station_map.csv", index=False)
WX["coverage"].to_csv(A4_DIR / "weather_coverage.csv")

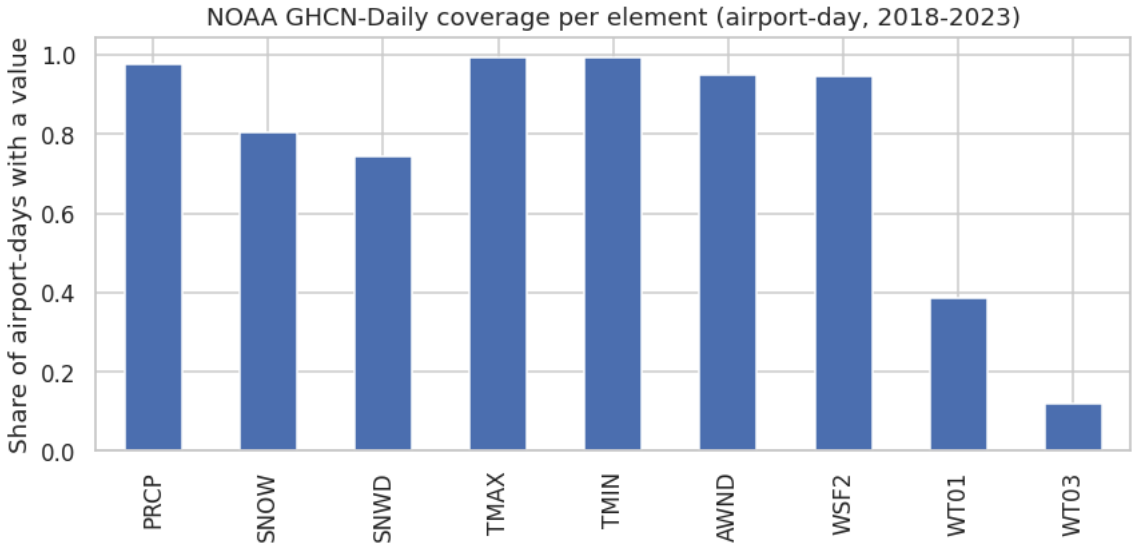
fig, ax = plt.subplots(figsize=(8, 4))
WX["coverage"].iloc[:, 0].plot.bar(ax=ax, color="#4C72B0")
ax.set_ylabel("Share of airport-days with a value")
ax.set_title("NOAA GHCN-Daily coverage per element (airport-day, 2018-2023)")
save_fig(fig, "22_weather_coverage")
```

```
379 airports, 43,135 candidate stations, 132,216 station-element rows

  [checkpoint saved]    a4_weather_airport_day
```

	PRCP	SNOW	SNWD	TMAX	TMIN	AWND	WSF2	WT01	WT03
airport-day coverage	0.975013	0.804011	0.745471	0.995163	0.995084	0.948229	0.946202	0.386556	0.120002

```
Nearest PRCP station: median 0.8 km, 95th pct 2.8 km, 1 airports with no PRCP station within 50 km
[saved] 22_weather_coverage.png
```



18.1 Assemble the A4 modelling table

Only the years needed for the production fold (2018, 2019, 2022 → 2023) are kept. The large A2 objects and the original `df` are released right after the slice is taken, to leave room in RAM (run this cell once per session; after a runtime restart, run the notebook from the top); re-running the earlier loading cell restores them from their checkpoints whenever they are needed again.

```
A4_BASE_NUMERIC = ["crs_dep_hour", "day_of_week", "is_weekend", "month", "is_holiday", "distance",
                   "carrier_delay_rate_7d", "origin_delay_rate_7d", "origin_degree_centrality"]
A4_BASE_CATEGORICAL = ["carrier", "origin_hub_tier"]
A4_V1_FEATURES = A4_BASE_NUMERIC + A4_BASE_CATEGORICAL          # same 11 inputs as A2, leakage-safe definitions
A4_WX_PRIOR = ["o_prcp_d1", "o_snow_d1", "o_snwd_d1", "o_tmax_d1", "o_tmin_d1", "o_awnd_d1", "o_wsf2_d1",
               "o_thunder_d1", "o_fog_d1", "o_prcp_3d",
               "d_prcp_d1", "d_snow_d1", "d_awnd_d1", "d_wsf2_d1", "d_thunder_d1"]
A4_V2_FEATURES = A4_V1_FEATURES + A4_WX_PRIOR
A4_WX_SAME_DAY = ["o_prcp_d0", "o_snow_d0", "o_awnd_d0", "o_wsf2_d0", "o_thunder_d0",
                  "d_prcp_d0", "d_snow_d0", "d_awnd_d0", "d_wsf2_d0", "d_thunder_d0"]   # NOT deployable (see 18
A4_TIERS = ["major", "medium", "small"]


def _lag_lookup(arr, a_idx, t_idx, lag):
    """arr[airport, day - lag]; NaN when the lagged day is before the data start."""
    out = np.full(len(t_idx), np.nan, dtype=np.float32)
    src = t_idx - lag
    ok = src >= 0
    out[ok] = arr[a_idx[ok], src[ok]]
    return out


def _window_sum_prior(arr, days=3):
    """sum of arr over days [t-days, t-1]; NaN if none of those days has an observation."""
    v = np.isfinite(arr)
    S = np.concatenate([np.zeros((arr.shape[0], 1)), np.cumsum(np.nan_to_num(arr), axis=1)], axis=1)
    V = np.concatenate([np.zeros((arr.shape[0], 1)), np.cumsum(v, axis=1)], axis=1)
    t = np.arange(arr.shape[1]); lo = np.maximum(t - days, 0)
    out = (S[:, t] - S[:, lo]).astype(np.float32)
    out[(V[:, t] - V[:, lo]) == 0] = np.nan
    return out


A4_PRCP_3D = _window_sum_prior(WX["W"]["PRCP"], 3)


def weather_features(frame, same_day=False):
    """Weather inputs for each flight, looked up from the airport-day tables. Prior-day (d-1) by default."""
    o_ap = np.array([A4_AP_INDEX.get(c, -1) for c in frame["origin"].cat.categories])[frame["origin"].cat.codes.
    d_ap = np.array([A4_AP_INDEX.get(c, -1) for c in frame["destination"].cat.categories])[frame["destination"].
    t = _day_index(frame["flight_date"])
```

```
        W, out = WX["W"], {}

        def look(el, ap, lag):
            res = np.full(len(t), np.nan, dtype=np.float32)
            ok = ap >= 0
            res[ok] = _lag_lookup(W[el], ap[ok], t[ok], lag)
            return res

        if not same_day:
            for name, el in [("o_prcp_d1", "PRCP"), ("o_snow_d1", "SNOW"), ("o_snwd_d1", "SNWD"), ("o_tmax_d1", "TMA
                             ("o_tmin_d1", "TMIN"), ("o_awnd_d1", "AWND"), ("o_wsf2_d1", "WSF2"), ("o_thunder_d1", "
                             ("o_fog_d1", "WT01")]:
                out[name] = look(el, o_ap, 1)
            res = np.full(len(t), np.nan, dtype=np.float32); ok = o_ap >= 0
            res[ok] = A4_PRCP_3D[o_ap[ok], t[ok]]
            out["o_prcp_3d"] = res
            for name, el in [("d_prcp_d1", "PRCP"), ("d_snow_d1", "SNOW"), ("d_awnd_d1", "AWND"), ("d_wsf2_d1", "WSF
                             ("d_thunder_d1", "WT03")]:
                out[name] = look(el, d_ap, 1)
        else:
            for name, el in [("o_prcp_d0", "PRCP"), ("o_snow_d0", "SNOW"), ("o_awnd_d0", "AWND"), ("o_wsf2_d0", "WSF
                             ("o_thunder_d0", "WT03")]:
                out[name] = look(el, o_ap, 0)
            for name, el in [("d_prcp_d0", "PRCP"), ("d_snow_d0", "SNOW"), ("d_awnd_d0", "AWND"), ("d_wsf2_d0", "WSF
                             ("d_thunder_d0", "WT03")]:
                out[name] = look(el, d_ap, 0)
        return pd.DataFrame(out, index=frame.index)


    if "d4" in globals() and all(c in d4.columns for c in A4_V2_FEATURES):
        print("A4 modelling table already built in this session -- skipping (the A2 `df` was released after it was s
    else:
        t0 = time.time()
        keep_cols = ["flight_date", "year", "carrier", "origin", "destination", "crs_dep_hour", "day_of_week", "is_w
                     "is_holiday", "distance", "arr_del15", "arr_delay_minutes", "origin_lat", "origin_lon"]
        m_years = df["year"].isin(A4_TRAIN_YEARS + [A4_TEST_YEAR]).to_numpy()
        d4 = df.loc[m_years, keep_cols].reset_index(drop=True)
        # fix the category sets to the six-year sets used for the rolling tables, so codes line up exactly
        d4["carrier"] = d4["carrier"].astype("category").cat.set_categories(CARR_CATS)
        d4["origin"] = d4["origin"].astype("category").cat.set_categories(ORIG_CATS)
        d4["destination"] = _as_cat(d4["destination"])
        for c in ["crs_dep_hour", "day_of_week", "is_weekend", "month", "is_holiday", "distance", "arr_delay_minutes
            d4[c] = d4[c].astype("float32")
        d4["arr_del15"] = d4["arr_del15"].astype("int8")

        # ---- release memory held by the A2 analysis BEFORE adding features (all restorable from checkpoints) ----
        _released = []
        for _name in ["df", "test_prod_full", "X_train_prod", "y_train_prod", "X_test_prod", "X_tr_inner", "y_tr_inn
                      "y_val_inner", "train_delayed", "test_delayed", "reg_tr_inner", "reg_val_inner", "X_shap", "sh
                      "fold_predictions", "walk_forward_results", "corr_sample", "monthly_carrier", "monthly_carrier
                      "sun_data", "delayed_only", "df_sample", "X_sample"]:
            if _name in globals():
                globals().pop(_name)
                _released.append(_name)
        if "final_models" in globals():                       # keep metrics + probabilities, drop the big fitted pip
            final_models = {k: (None, v[1], v[2]) for k, v in final_models.items()}
            final_logreg = final_rf = final_xgb = final_lgbm = None
        gc.collect()
        print("Released from the A2 session:", ", ".join(_released))

        # Rolling carrier / origin delay rates: flight-weighted, calendar window [d-7, d-1]
        t_idx = _day_index(d4["flight_date"])
        m_tr = d4["year"].isin(A4_TRAIN_YEARS).to_numpy()
        fill_rate = float(d4.loc[m_tr, "arr_del15"].mean())                # training-window mean for the very firs
        for col, rate, key in [("carrier_delay_rate_7d", CARR_RATE, "carrier"), ("origin_delay_rate_7d", ORIG_RATE,
            codes = d4[key].cat.codes.to_numpy()
            assert codes.min() >= 0
            v = rate[codes, t_idx]
            print(f"{col}: {np.isnan(v).mean():.4%} of rows have no flights in the prior 7 days -> filled with train
            d4[col] = np.where(np.isnan(v), fill_rate, v).astype("float32")

        # Graph features from the training window only (section 17)
        o_codes = d4["origin"].cat.codes.to_numpy()
        cats = list(d4["origin"].cat.categories)
        d4["origin_degree_centrality"] = np.array([A4_DEGREE.get(c, 0.0) for c in cats], dtype=np.float32)[o_codes]
        tier_arr = np.array([A4_TIERS.index(A4_TIER.get(c, "small")) for c in cats])[o_codes]
        d4["origin_hub_tier"] = pd.Categorical.from_codes(tier_arr, categories=A4_TIERS)

        # Weather (prior day)
        wx_prior = weather_features(d4)
        for c in wx_prior.columns:
            d4[c] = wx_prior[c]
```

```
    del wx_prior
    gc.collect()
    print(f"A4 modelling table built in {time.time() - t0:.0f}s")

print(f"A4 modelling table: {len(d4):,} rows x {d4.shape[1]} cols, {d4.memory_usage(deep=True).sum() / 1e9:.2f}
fill_rate = float(d4.loc[d4["year"].isin(A4_TRAIN_YEARS), "arr_del15"].mean())
print("Missing share per weather feature:")
print(d4[A4_WX_PRIOR].isna().mean().round(4).to_string())

m_train = d4["year"].isin(A4_TRAIN_YEARS).to_numpy()
m_test = (d4["year"] == A4_TEST_YEAR).to_numpy()
val_cut = d4.loc[m_train, "flight_date"].quantile(1 - A4_VAL_FRACTION)
m_inner = m_train & (d4["flight_date"] < val_cut).to_numpy()
m_val = m_train & (d4["flight_date"] >= val_cut).to_numpy()
print(f"train {m_train.sum():,} (inner {m_inner.sum():,} / validation {m_val.sum():,}, validation starts {val_cu
```

```
Released from the A2 session: df, test_prod_full, X_train_prod, y_train_prod, X_test_prod, X_tr_inner, y_tr_inner
carrier_delay_rate_7d: 0.0645% of rows have no flights in the prior 7 days -> filled with training mean 0.197
origin_delay_rate_7d: 0.0646% of rows have no flights in the prior 7 days -> filled with training mean 0.197
A4 modelling table built in 22s
A4 modelling table: 27,606,000 rows x 34 cols, 3.45 GB
Missing share per weather feature:
o_prcp_d1       0.0033
o_snow_d1       0.1205
o_snwd_d1       0.1337
o_tmax_d1       0.0008
o_tmin_d1       0.0008
o_awnd_d1       0.0029
o_wsf2_d1       0.0030
o_thunder_d1    0.0006
o_fog_d1        0.0006
o_prcp_3d       0.0029
d_prcp_d1       0.0033
d_snow_d1       0.1206
d_awnd_d1       0.0029
d_wsf2_d1       0.0030
d_thunder_d1    0.0006
train 20,865,861 (inner 17,718,989 / validation 3,146,872, validation starts 2022-07-11) | test 6,740,139
```

19. Leakage tests and the feature availability table

Automated checks that fail loudly if any input could contain information from the flight's own day or later:

1. dense daily counts equal a direct row count (spot-checked on raw rows);
2. the rolling rate equals an explicit loop over calendar days `[d−7, d−1]`;
3. **perturbation test** — flipping every outcome on day *d* must not change the feature on day *d* (but must change day *d+1*);
4. graph features contain no airport that was unseen in training;
5. prior-day weather at day *d* equals the observation of *d-1*, and changing day *d*'s weather does not alter it;
6. no feature is a function of the targets.

```
tests = []
rng = np.random.RandomState(RANDOM_STATE)

# 1 + 2: dense tables vs explicit loops
ok2 = True
Nk, Dk = ORIG_N, ORIG_D
for _ in range(2000):
    k, t = rng.randint(Nk.shape[0]), rng.randint(1, A4_T)
    lo = max(t - A4_WINDOW_DAYS, 0)
    n_loop = sum(Nk[k, u] for u in range(lo, t)); d_loop = sum(Dk[k, u] for u in range(lo, t))
    expected = d_loop / n_loop if n_loop else np.nan
    got = ORIG_RATE[k, t]
    if not ((np.isnan(expected) and np.isnan(got)) or abs(expected - got) < 1e-6):
        ok2 = False
tests.append(("Rolling rate == explicit loop over [d-7, d-1] (2,000 random origin-days)", ok2))

# 3: perturbation (flip all outcomes on one day for a busy airport)
k = int(np.argmax(ORIG_N.sum(axis=1)))
t = 1000
D2 = ORIG_D.copy(); D2[k, t] = ORIG_N[k, t] - ORIG_D[k, t]
r_orig, _ = _rolling_rate(ORIG_N, ORIG_D); r_pert, _ = _rolling_rate(ORIG_N, D2)
same_day_unchanged = np.isclose(r_orig[k, t], r_pert[k, t], equal_nan=True)
next_day_changes = (not np.isclose(r_orig[k, t + 1], r_pert[k, t + 1])) if ORIG_D[k, t] * 2 != ORIG_N[k, t] else
tests.append(("Perturbing day-d outcomes leaves the day-d feature unchanged", bool(same_day_unchanged)))
tests.append(("... but does reach day d+1 (the window is not vacuous)", bool(next_day_changes)))

# 4: graph features
unseen_in_train = [c for c in d4["origin"].cat.categories if c not in A4_TIER.index]
unseen_rows = d4["origin"].isin(unseen_in_train).to_numpy()
tests.append(("Airports unseen in training get fallback tier 'small' / centrality 0",
```

```
              bool((d4.loc[unseen_rows, "origin_hub_tier"] == "small").all() and (d4.loc[unseen_rows, "origin_de
tests.append(("origin_degree_centrality within [0, 1]",
              bool(d4["origin_degree_centrality"].between(0, 1).all())))

# 5: weather timing
o_ap = np.array([A4_AP_INDEX.get(c, -1) for c in d4["origin"].cat.categories])[d4["origin"].cat.codes.to_numpy()
sample = rng.choice(np.flatnonzero(o_ap >= 0), size=5000, replace=False)
tt = _day_index(d4["flight_date"].iloc[sample])
direct = WX["W"]["PRCP"][o_ap[sample], np.maximum(tt - 1, 0)]
got = d4["o_prcp_d1"].to_numpy()[sample]
valid = tt >= 1
tests.append(("o_prcp_d1 equals the PRCP observation of day d-1 (5,000 random flights)",
              bool(np.allclose(direct[valid], got[valid], equal_nan=True))))
Wtest = WX["W"]["PRCP"].copy(); a_, t_ = int(o_ap[sample[0]]), int(tt[0])
before = _lag_lookup(Wtest, np.array([a_]), np.array([t_]), 1)[0]
Wtest[a_, t_] = 9999.0                                    # change *same-day* weather
after = _lag_lookup(Wtest, np.array([a_]), np.array([t_]), 1)[0]
tests.append(("Changing day-d weather does not change the prior-day feature", bool(np.isclose(before, after, equ

# 6: no target-derived inputs
tests.append(("No input column is a target or target-derived (arr_*)",
              bool(not any(c.startswith("arr_") for c in A4_V2_FEATURES))))

leak_df = pd.DataFrame(tests, columns=["check", "passed"])
display(leak_df)
leak_df.to_csv(A4_DIR / "leakage_audit.csv", index=False)
assert leak_df["passed"].all(), "A leakage test FAILED -- do not continue"
print("All leakage checks passed.")
```

	check	passed
0	Rolling rate == explicit loop over [d-7, d-1] ...	True
1	Perturbing day-d outcomes leaves the day-d fea...	True
2	... but does reach day d+1 (the window is not ...	True
3	Airports unseen in training get fallback tier ...	True
4	origin_degree_centrality within [0, 1]	True
5	o_prcp_d1 equals the PRCP observation of day d...	True
6	Changing day-d weather does not change the pri...	True
7	No input column is a target or target-derived ...	True

```
All leakage checks passed.
```

Next steps: New interactive sheet

```
availability = pd.DataFrame([
    ("crs_dep_hour, day_of_week, is_weekend, month", "BTS schedule", "Published schedule, weeks ahead", "Determi
    ("is_holiday", "`holidays` calendar", "Known years ahead", "Deterministic calendar lookup", "Yes"),
    ("distance", "BTS / route table", "Known when the flight is scheduled", "Route constant", "Yes"),
    ("carrier", "BTS", "Known when the flight is scheduled", "Categorical", "Yes"),
    ("carrier_delay_rate_7d", "BTS outcomes", "Available after the overnight refresh on flight day", "Flight-wei
    ("origin_delay_rate_7d", "BTS outcomes", "Available at 00:00 on flight day", "Same, grouped by origin airpor
    ("origin_degree_centrality, origin_hub_tier", "BTS route history", "Fixed at training time", "Computed from
    ("o_*_d1, d_*_d1 (prior-day weather)", "NOAA GHCN-Daily", "Day d-1 summary is normally posted within about a
    ("o_prcp_3d", "NOAA GHCN-Daily", "Same", "Precipitation total over [d-3, d-1]", "Yes"),
    ("o_*_d0, d_*_d0 (same-day weather)", "NOAA GHCN-Daily", "Only after day d has finished", "Used ONLY in the
    ("arr_del15, arr_delay_minutes", "BTS", "After the flight lands", "Targets only", "Never an input"),
], columns=["feature(s)", "source", "when it becomes available", "construction / safeguard", "usable at predicti
availability.to_csv(A4_DIR / "feature_availability.csv", index=False)
pd.set_option("display.max_colwidth", 80)
availability
```

	feature(s)	source	when it becomes available	construction / safeguard	usable at prediction time?
0	crs_dep_hour, day_of_week, is_weekend, month	BTS schedule	Published schedule, weeks ahead	Deterministic function of scheduled time	Yes
1	is_holiday	`holidays` calendar	Known years ahead	Deterministic calendar lookup	Yes
2	distance	BTS / route table	Known when the flight is scheduled	Route constant	Yes
3	carrier	BTS	Known when the flight is scheduled	Categorical	Yes
4	carrier_delay_rate_7d	BTS outcomes	Available after the overnight refresh on flight day	Flight-weighted arr_del15 over calendar days [d-7, d-1]. Caveat: a late-even...	Yes (needs a nightly refresh)
5	origin_delay_rate_7d	BTS outcomes	Available at 00:00 on flight day	Same, grouped by origin airport	Yes (needs a nightly refresh)
6	origin_degree_centrality, origin_hub_tier	BTS route history	Fixed at training time	Computed from training years only; unseen airports -> fallback	Yes
7	o_*_d1, d_*_d1 (prior-day weather)	NOAA GHCN-Daily	Day d-1 summary is normally posted within about a day; confirm the latency b...	Nearest station within 50 km, previous calendar day	Yes (needs a daily NOAA pull)

Next steps: New interactive sheet

20. Ablation: how do the corrections and the weather additions change performance?

Same tuned LightGBM hyper-parameters, same production fold (train 2018 + 2019 + 2022, test 2023), balanced class weights as in A2.

Variant	Features
A2 (reference)	the published A2 result (full-six-year hub features, row-based rolling rates) — read from its checkpoint, not re-run
V1	same 11 inputs, leakage-safe definitions (calendar window, training-window graph features)
V2	V1 + prior-day NOAA weather (deployable)
V3	V2 + *same-day* NOAA weather — **upper bound only, not deployable**

```
def _fit_clf(X, y):
    return LGBMClassifier(**A4_CLF_PARAMS, class_weight="balanced", n_jobs=-1, random_state=RANDOM_STATE, verbos


def _metrics(y, p, thr=0.5):
    yhat = (p >= thr).astype(int)
    return {"auc_roc": roc_auc_score(y, p), "avg_precision": average_precision_score(y, p), "f1": f1_score(y, yh
            "precision": precision_score(y, yhat, zero_division=0), "recall": recall_score(y, yhat), "brier": br


y_tr_all = d4.loc[m_train, "arr_del15"].to_numpy()
y_te = d4.loc[m_test, "arr_del15"].to_numpy()


def run_variant(tag, features, frame=None):
    frame = d4 if frame is None else frame

    def _compute():
        t0 = time.time()
        model = _fit_clf(frame.loc[m_train, features], y_tr_all)
        p = model.predict_proba(frame.loc[m_test, features])[:, 1].astype(np.float32)
        rec = {"metrics": _metrics(y_te, p), "proba": p, "seconds": round(time.time() - t0)}
        del model
        gc.collect()
        return rec
    rec = ckpt_step(f"a4_abl_{tag}", _compute)
    m = rec["metrics"]
    print(f"  [{tag}] AUC={m['auc_roc']:.4f} AP={m['avg_precision']:.4f} F1@0.5={m['f1']:.4f} Brier={m['brier']:
    return rec


abl = {}
abl["V1_leakage_safe"] = run_variant("v1", A4_V1_FEATURES)
abl["V2_plus_prior_day_weather"] = run_variant("v2", A4_V2_FEATURES)

# V3: same-day weather -- built on the fly, never stored in d4 (not a deployable feature set)
if not ckpt_exists("a4_abl_v3"):
    _same = weather_features(d4, same_day=True)
    for _c in _same.columns:
```

```
        d4[_c] = _same[_c]                                  # temporary columns, dropped again below
    del _same
    abl["V3_same_day_weather_UPPER_BOUND"] = run_variant("v3", A4_V2_FEATURES + A4_WX_SAME_DAY)
    d4.drop(columns=A4_WX_SAME_DAY, inplace=True)
    gc.collect()
else:
    abl["V3_same_day_weather_UPPER_BOUND"] = run_variant("v3", A4_V2_FEATURES + A4_WX_SAME_DAY)

# A2 reference from its checkpoint (published numbers; probabilities only for average precision)
a2_row = None
if "final_models" in globals() and "lightgbm_tuned" in final_models:
    _m, _p = final_models["lightgbm_tuned"][1], final_models["lightgbm_tuned"][2]
    a2_row = dict(_m)
    a2_row["avg_precision"] = average_precision_score(y_test_prod, _p) if "y_test_prod" in globals() else np.nan
rows = []
if a2_row:
    rows.append({"variant": "A2 (reference, published)", **a2_row})
rows += [{"variant": k, **v["metrics"]} for k, v in abl.items()]
ablation_df = pd.DataFrame(rows)[["variant", "auc_roc", "avg_precision", "f1", "precision", "recall", "brier"]]
ablation_df.to_csv(A4_DIR / "a4_ablation_results.csv", index=False)
display(ablation_df.style.format({c: "{:.4f}" for c in ablation_df.columns if c != "variant"}))

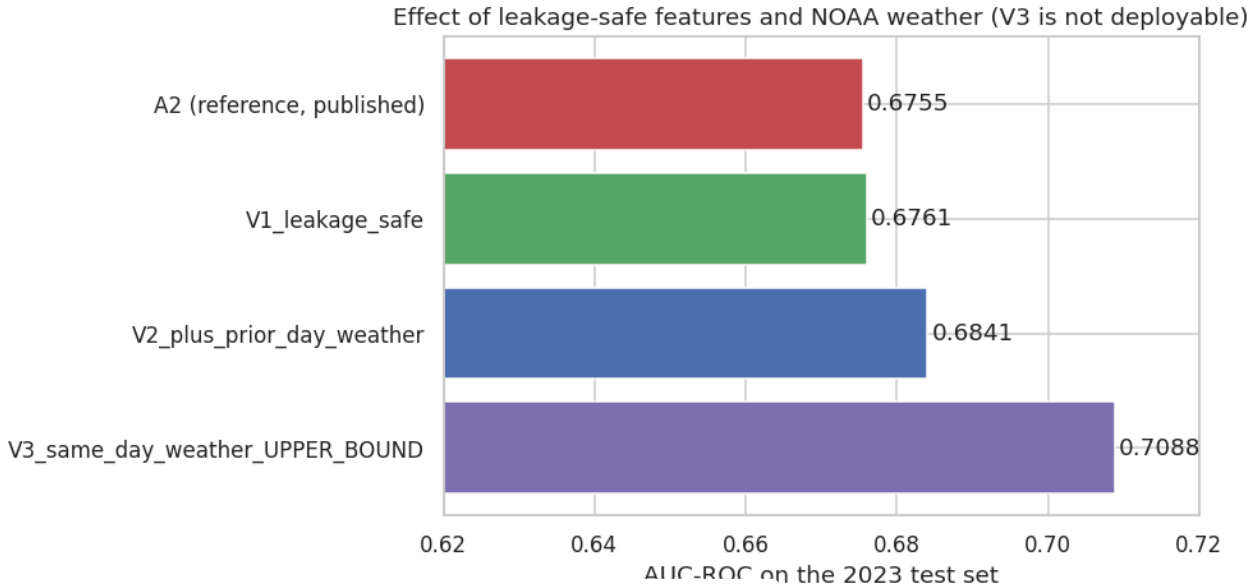
# Is V2 > V1 beyond sampling noise?  paired bootstrap on the 2023 test set
bi = np.random.RandomState(RANDOM_STATE)
deltas = []
for _ in range(30):
    ii = bi.randint(0, len(y_te), size=1_000_000)
    deltas.append(roc_auc_score(y_te[ii], abl["V2_plus_prior_day_weather"]["proba"][ii]) - roc_auc_score(y_te[ii
print(f"AUC(V2) - AUC(V1): mean {np.mean(deltas):+.4f}, 95% interval [{np.percentile(deltas, 2.5):+.4f}, {np.per

fig, ax = plt.subplots(figsize=(9, 4.5))
ax.barh(ablation_df["variant"][::-1], ablation_df["auc_roc"][::-1], color=["#C44E52", "#55A868", "#4C72B0", "#81
ax.set_xlim(0.62, max(0.72, ablation_df["auc_roc"].max() + 0.01))
for i, v in enumerate(ablation_df["auc_roc"][::-1]):
    ax.text(v + 0.0005, i, f"{v:.4f}", va="center")
ax.set_xlabel("AUC-ROC on the 2023 test set")
ax.set_title("Effect of leakage-safe features and NOAA weather (V3 is not deployable)")
save_fig(fig, "23_ablation_auc")
```

```
  [checkpoint saved]    a4_abl_v1
  [v1] AUC=0.6761 AP=0.3570 F1@0.5=0.4080 Brier=0.2314
  [checkpoint saved]    a4_abl_v2
  [v2] AUC=0.6841 AP=0.3645 F1@0.5=0.4147 Brier=0.2264
  [checkpoint saved]    a4_abl_v3
  [v3] AUC=0.7088 AP=0.4052 F1@0.5=0.4372 Brier=0.2142
```

	variant	auc_roc	avg_precision	f1	precision	recall	brier
0	A2 (reference, published)	0.6755	0.3555	0.4072	0.2974	0.6455	0.2316
1	V1_leakage_safe	0.6761	0.3570	0.4080	0.2985	0.6442	0.2314
2	V2_plus_prior_day_weather	0.6841	0.3645	0.4147	0.3076	0.6364	0.2264
3	V3_same_day_weather_UPPER_BOUND	0.7088	0.4052	0.4372	0.3378	0.6197	0.2142

```
AUC(V2) - AUC(V1): mean +0.0080, 95% interval [+0.0076, +0.0085] (30 paired bootstraps of 1M flights)
[saved] 23_ablation_auc.png
```



21. From a score to a decision: calibration, thresholds and alert workload

With balanced class weights the raw LightGBM score is a *ranking* signal, not a probability (A2 flagged 44.5% of flights at 0.5 with precision ≈ 0.30). An operations user needs (i) a probability that means what it says, (ii) the threshold used, and (iii) how many alerts that produces.

- The **production model** is V2 fitted on the first 85% of the training dates; the **last 15%** (late 2022) is a held-out *validation* set.
- A calibrator (Platt or isotonic, chosen on held-out validation data) maps the raw score to a probability.
- **All thresholds are chosen on the validation set**, then applied once to the 2023 test set.

```
def _ece(y, p, bins=15):
    edges = np.unique(np.quantile(p, np.linspace(0, 1, bins + 1)))
    b = np.clip(np.digitize(p, edges[1:-1]), 0, len(edges) - 2)
    cnt = np.bincount(b); obs = np.bincount(b, weights=y) / np.maximum(cnt, 1); pred = np.bincount(b, weights=p)
    return float(np.sum(cnt * np.abs(obs - pred)) / len(y))


def _compute_prod():
    t0 = time.time()
    model = _fit_clf(d4.loc[m_inner, A4_V2_FEATURES], d4.loc[m_inner, "arr_del15"].to_numpy())
    p_val = model.predict_proba(d4.loc[m_val, A4_V2_FEATURES])[:, 1].astype(np.float32)
    p_te = model.predict_proba(d4.loc[m_test, A4_V2_FEATURES])[:, 1].astype(np.float32)
    return {"model": model, "p_val_raw": p_val, "p_test_raw": p_te, "seconds": round(time.time() - t0)}


prod = ckpt_step("a4_prod_inner", _compute_prod)
prod_model, p_val_raw, p_te_raw = prod["model"], prod["p_val_raw"], prod["p_test_raw"]
y_val = d4.loc[m_val, "arr_del15"].to_numpy()
print(f"Production model fitted on {m_inner.sum():,} rows; validation {len(y_val):,} rows (base rate {y_val.mean
      f"test base rate {y_te.mean():.3f}")


class _Platt:
    def __init__(self): self.lr = LogisticRegression(C=1e6, max_iter=500)
    def fit(self, p, y): self.lr.fit(self._z(p), y); return self
    def predict(self, p): return self.lr.predict_proba(self._z(p))[:, 1]
    @staticmethod
    def _z(p): p = np.clip(p, 1e-6, 1 - 1e-6); return np.log(p / (1 - p)).reshape(-1, 1)


class _Iso:
    def __init__(self): self.iso = IsotonicRegression(out_of_bounds="clip", y_min=0.0, y_max=1.0)
    def fit(self, p, y): self.iso.fit(p, y); return self
    def predict(self, p): return self.iso.predict(p)


# Choose the calibration method on held-out validation data (first half -> second half by date), then refit on a
_o = np.argsort(d4.loc[m_val, "flight_date"].to_numpy(), kind="stable")     # date order, regardless of row orde
h = len(y_val) // 2
choice = {}
for name, cls in [("platt", _Platt), ("isotonic", _Iso)]:
    choice[name] = brier_score_loss(y_val[_o[h:]], cls().fit(p_val_raw[_o[:h]], y_val[_o[:h]]).predict(p_val_raw
best = min(choice, key=choice.get)
calibrator = (_Platt if best == "platt" else _Iso)().fit(p_val_raw, y_val)
print("Held-out validation Brier by method:", {k: round(v, 5) for k, v in choice.items()}, "-> using", best)

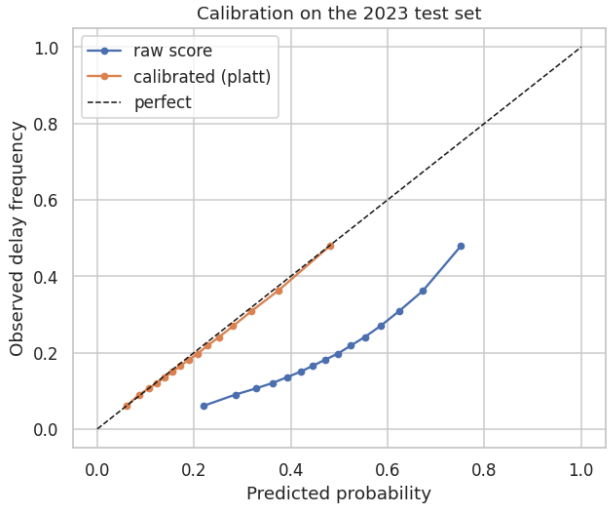
p_val = calibrator.predict(p_val_raw).astype(np.float32)
p_te = calibrator.predict(p_te_raw).astype(np.float32)
cal_rows = [{"version": "raw score (balanced class weights)", **_metrics(y_te, p_te_raw), "ECE": _ece(y_te, p_te
            {"version": f"calibrated ({best})", **_metrics(y_te, p_te, 0.5), "ECE": _ece(y_te, p_te)}]
cal_df = pd.DataFrame(cal_rows)
cal_df.to_csv(A4_DIR / "a4_calibration_summary.csv", index=False)
display(cal_df.style.format({c: "{:.4f}" for c in cal_df.columns if c != "version"}))

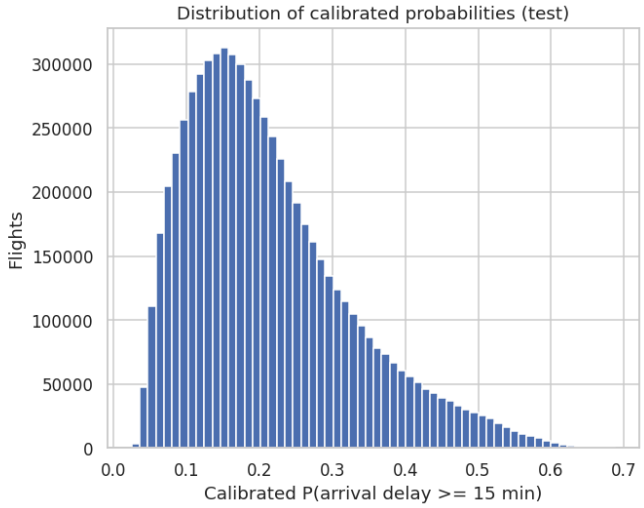
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for p, lab in [(p_te_raw, "raw score"), (p_te, f"calibrated ({best})")]:
    fp, mp = calibration_curve(y_te, p, n_bins=15, strategy="quantile")
    axes[0].plot(mp, fp, marker="o", markersize=4, label=lab)
axes[0].plot([0, 1], [0, 1], "k--", linewidth=1, label="perfect")
axes[0].set_xlabel("Predicted probability"); axes[0].set_ylabel("Observed delay frequency")
axes[0].set_title("Calibration on the 2023 test set"); axes[0].legend()
axes[1].hist(p_te, bins=60, color="#4C72B0")
axes[1].set_xlabel("Calibrated P(arrival delay >= 15 min)"); axes[1].set_ylabel("Flights")
axes[1].set_title("Distribution of calibrated probabilities (test)")
save_fig(fig, "24_calibration_before_after")
```

```
  [checkpoint saved]    a4_prod_inner
Production model fitted on 17,718,989 rows; validation 3,146,872 rows (base rate 0.208); test base rate 0.206
Held-out validation Brier by method: {'platt': np.float64(0.15343), 'isotonic': np.float64(0.15353)} -> using pla
```

	version	auc_roc	avg_precision	f1	precision	recall	brier	ECE
0	raw score (balanced class weights)	0.6824	0.3629	0.4132	0.3065	0.6339	0.2261	0.2697
1	calibrated (platt)	0.6824	0.3629	0.1041	0.5578	0.0574	0.1513	0.0063

```
[saved] 24_calibration_before_after.png
```






```
# ---- Operating points: chosen on VALIDATION data, evaluated once on the 2023 test set ------------------------
def _confusion(y, p, thr):
    yhat = p >= thr
    tp = int((yhat & (y == 1)).sum()); fp = int((yhat & (y == 0)).sum()); fn = int((~yhat & (y == 1)).sum())
    return tp, fp, fn, int((~yhat & (y == 0)).sum())


def _op_row(label, thr):
    tp, fp, fn, tn = _confusion(y_te, p_te, thr)
    prec = tp / max(tp + fp, 1); rec = tp / max(tp + fn, 1)
    flagged = (tp + fp) / len(y_te)
    return {"operating point": label, "threshold": thr, "flights flagged": flagged, "alerts per day (network)":
            "precision": prec, "recall": rec, "F1": 2 * prec * rec / max(prec + rec, 1e-12),
            "false alarms per 10 alerts": 10 * (1 - prec), "lift over base rate": prec / y_te.mean()}


grid = np.round(np.arange(0.05, 0.96, 0.01), 2)
val_f1 = []
val_cost = []
for thr in grid:
    tp, fp, fn, tn = _confusion(y_val, p_val, thr)
    pr = tp / max(tp + fp, 1); rc = tp / max(tp + fn, 1)
    val_f1.append(2 * pr * rc / max(pr + rc, 1e-12))
    val_cost.append(A4_COST_FN * fn + A4_COST_FP * fp)
thr_f1 = float(grid[int(np.argmax(val_f1))])
thr_cost = float(grid[int(np.argmin(val_cost))])
thr_cost_theory = A4_COST_FP / (A4_COST_FP + A4_COST_FN)           # optimal for a well-calibrated probability
ops = [_op_row("max F1 on validation", thr_f1),
       _op_row(f"min expected cost on validation (FN:FP = {A4_COST_FN:g}:{A4_COST_FP:g})", thr_cost)]
for b in A4_ALERT_BUDGETS:
    ops.append(_op_row(f"alert budget: top {b:.0%} of flights", float(np.quantile(p_val, 1 - b))))
# precision-target operating point (if reachable on validation)
for target in (0.40, 0.50):
    reachable = [t for t in grid if (lambda c: c[0] / max(c[0] + c[1], 1))(_confusion(y_val, p_val, t)) >= targe
    ops.append(_op_row(f"smallest threshold with validation precision >= {target:.0%}", float(reachable[0])) if
               {"operating point": f"precision >= {target:.0%} not reachable on validation", "threshold": np.nan
tp, fp, fn, tn = _confusion(y_te, p_te_raw, 0.5)
ops.insert(0, {"operating point": "A2 reference: raw score >= 0.5", "threshold": 0.5, "flights flagged": (tp + f
               "alerts per day (network)": (tp + fp) / 365.0, "precision": tp / max(tp + fp, 1), "recall": tp /
               "F1": 2 * tp / max(2 * tp + fp + fn, 1), "false alarms per 10 alerts": 10 * fp / max(tp + fp, 1),
               "lift over base rate": (tp / max(tp + fp, 1)) / y_te.mean()})
ops_df = pd.DataFrame(ops)
ops_df.to_csv(A4_DIR / "a4_operating_points.csv", index=False)
print(f"Theoretical cost-optimal threshold for a calibrated probability: {thr_cost_theory:.2f}")
```

```
display(ops_df.style.format({c: "{:.3f}" for c in ops_df.columns if c != "operating point"}, na_rep="-"))
A4_THRESHOLDS = {"max_f1": thr_f1, "min_cost": thr_cost, "cost_theory": thr_cost_theory,
                 **{f"budget_{int(b * 100)}pct": float(np.quantile(p_val, 1 - b)) for b in A4_ALERT_BUDGETS}}

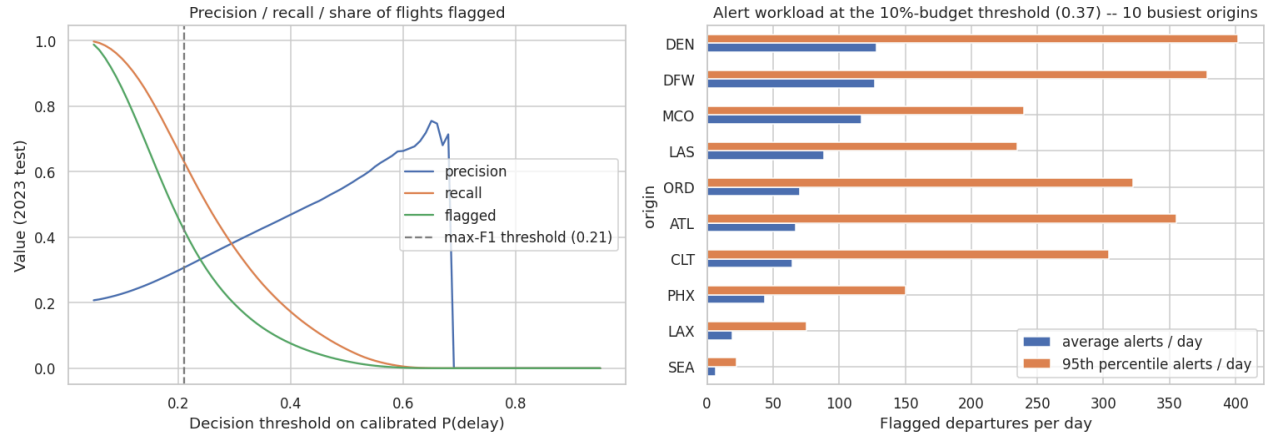
# Curves and workload per airport
curve = []
for thr in grid:
    tp, fp, fn, tn = _confusion(y_te, p_te, thr)
    curve.append({"threshold": thr, "precision": tp / max(tp + fp, 1), "recall": tp / max(tp + fn, 1), "flagged"
curve = pd.DataFrame(curve)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for col, c in [("precision", "#4C72B0"), ("recall", "#DD8452"), ("flagged", "#55A868")]:
    axes[0].plot(curve["threshold"], curve[col], label=col, color=c)
axes[0].axvline(thr_f1, color="grey", linestyle="--", label=f"max-F1 threshold ({thr_f1:.2f})")
axes[0].set_xlabel("Decision threshold on calibrated P(delay)"); axes[0].set_ylabel("Value (2023 test)")
axes[0].set_title("Precision / recall / share of flights flagged"); axes[0].legend()

thr_b = A4_THRESHOLDS["budget_10pct"]
daily = d4.loc[m_test, ["origin", "flight_date"]].assign(flag=(p_te >= thr_b))
top = daily["origin"].value_counts().head(10).index
alerts = daily[daily["origin"].isin(top)].groupby(["origin", "flight_date"], observed=True)["flag"].sum().groupb
alerts.columns = ["average alerts / day", "95th percentile alerts / day"]
alerts.sort_values("average alerts / day").plot.barh(ax=axes[1], color=["#4C72B0", "#DD8452"])
axes[1].set_title(f"Alert workload at the 10%-budget threshold ({thr_b:.2f}) -- 10 busiest origins")
axes[1].set_xlabel("Flagged departures per day")
save_fig(fig, "25_threshold_tradeoff_and_workload")
alerts.to_csv(A4_DIR / "a4_alert_workload_top_airports.csv")
```

```
Theoretical cost-optimal threshold for a calibrated probability: 0.25
```

	operating point	threshold	flights flagged	alerts per day (network)	precision	recall	F1	false alarms per 10 alerts	lift over base rate
0	A2 reference: raw score >= 0.5	0.500	0.425	7855.082	0.306	0.634	0.413	6.935	1.490
1	max F1 on validation	0.210	0.424	7826.304	0.307	0.632	0.413	6.931	1.492
2	min expected cost on validation (FN:FP = 3:1)	0.250	0.303	5601.244	0.342	0.504	0.407	6.581	1.662
3	alert budget: top 5% of flights	0.431	0.054	996.573	0.495	0.130	0.206	5.046	2.409
4	alert budget: top 10% of flights	0.367	0.105	1935.458	0.442	0.225	0.298	5.583	2.148
5	alert budget: top 20% of flights	0.293	0.209	3864.901	0.379	0.386	0.382	6.211	1.843
6	smallest threshold with validation precision >= 40%	0.310	0.180	3316.036	0.394	0.344	0.367	6.064	1.914
7	smallest threshold with validation precision >= 50%	0.440	0.049	898.910	0.503	0.119	0.192	4.974	2.444

```
[saved] 25_threshold_tradeoff_and_workload.png
```



22. Delay duration: an interval with an honest reliability statement

A2 found $R^2 \approx 0$ for the minutes regressor. A single "≈ N min" figure therefore overstates what the data support. Here the duration model is re-fitted (V2 features) as three **quantile regressions** (10th / 50th / 90th percentile) so the dashboard can show a *range* and the coverage can be checked, and the result is compared with a naive baseline that ignores all features.

Scope: like A2, the duration model is trained and evaluated on flights that *were* delayed (arrival delay >= 15 min), so its range answers "if this flight is delayed, how long?". It is not shown for flights the classifier does not flag.

```
m_tr_delayed = m_train & (d4["arr_del15"] == 1).to_numpy()
m_te_delayed = m_test & (d4["arr_del15"] == 1).to_numpy()


def _compute_qreg():
    t0 = time.time()
    X, y = d4.loc[m_tr_delayed, A4_V2_FEATURES], d4.loc[m_tr_delayed, "arr_delay_minutes"].to_numpy()
    models = {q: LGBMRegressor(objective="quantile", alpha=q, **A4_REG_PARAMS, n_jobs=-1, random_state=RANDOM_ST
              for q in A4_QUANTILES}
    return {"models": models, "naive": {q: float(np.quantile(y, q)) for q in A4_QUANTILES}, "seconds": round(tim


qreg = ckpt_step("a4_duration_quantile", _compute_qreg)
Xq, yq = d4.loc[m_te_delayed, A4_V2_FEATURES], d4.loc[m_te_delayed, "arr_delay_minutes"].to_numpy()
qpred = np.sort(np.column_stack([qreg["models"][q].predict(Xq) for q in A4_QUANTILES]), axis=1)     # enforce q1


def _pinball(y, p, q): d = y - p; return float(np.mean(np.maximum(q * d, (q - 1) * d)))

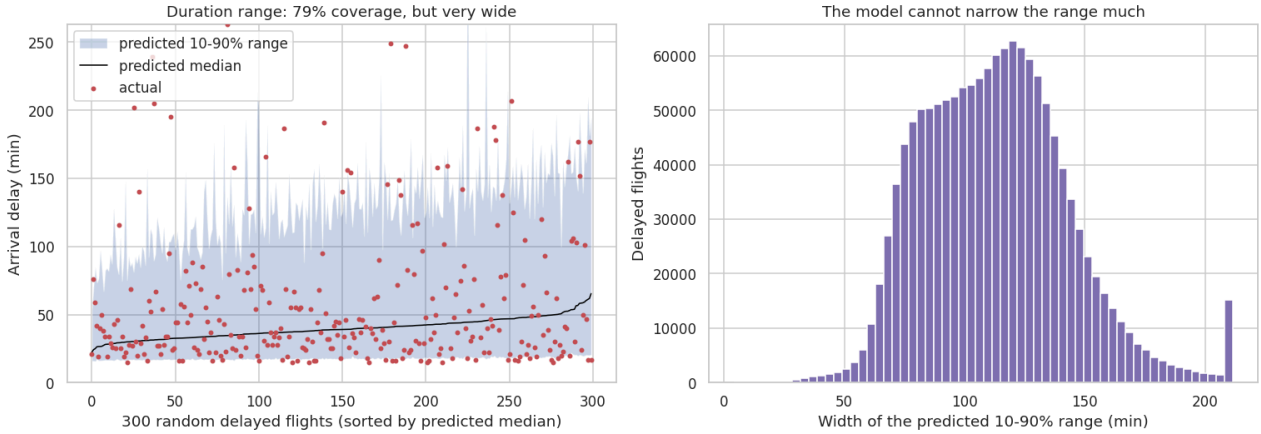

rows = []
for i, q in enumerate(A4_QUANTILES):
    mdl, base = _pinball(yq, qpred[:, i], q), _pinball(yq, np.full_like(yq, qreg["naive"][q]), q)
    rows.append({"quantile": q, "pinball loss (model)": mdl, "pinball loss (naive quantile)": base, "skill vs na
dur_df = pd.DataFrame(rows)
cover = float(np.mean((yq >= qpred[:, 0]) & (yq <= qpred[:, 2])))
width = qpred[:, 2] - qpred[:, 0]
print(f"Nominal 80% interval [q10, q90]: empirical coverage {cover:.1%}; median width {np.median(width):.0f} min
      f"(IQR {np.percentile(width, 25):.0f}-{np.percentile(width, 75):.0f})")
print(f"Median-model MAE {mean_absolute_error(yq, qpred[:, 1]):.1f} min | naive (training median) MAE "
      f"{mean_absolute_error(yq, np.full_like(yq, qreg['naive'][0.5])):.1f} min | median-model R2 {r2_score(yq,
display(dur_df.style.format({c: "{:.3f}" for c in dur_df.columns}))
dur_df.assign(coverage_80pct=cover, median_interval_width_min=float(np.median(width))).to_csv(A4_DIR / "a4_durat

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
s = np.random.RandomState(RANDOM_STATE).choice(len(yq), 300, replace=False); s = s[np.argsort(qpred[s, 1])]
axes[0].fill_between(range(len(s)), qpred[s, 0], qpred[s, 2], alpha=0.3, label="predicted 10-90% range")
axes[0].plot(qpred[s, 1], color="black", linewidth=1, label="predicted median")
axes[0].scatter(range(len(s)), yq[s], s=8, color="#C44E52", label="actual", zorder=3)
axes[0].set_ylim(0, np.percentile(yq[s], 97)); axes[0].set_xlabel("300 random delayed flights (sorted by predict
axes[0].set_ylabel("Arrival delay (min)"); axes[0].set_title(f"Duration range: {cover:.0%} coverage, but very wi
axes[1].hist(np.clip(width, 0, np.percentile(width, 99)), bins=60, color="#8172B2")
axes[1].set_xlabel("Width of the predicted 10-90% range (min)"); axes[1].set_ylabel("Delayed flights")
axes[1].set_title("The model cannot narrow the range much")
save_fig(fig, "26_duration_intervals")
```

```
  [checkpoint saved]    a4_duration_quantile
Nominal 80% interval [q10, q90]: empirical coverage 78.8%; median width 112 min (IQR 90–132)
Median-model MAE 43.7 min | naive (training median) MAE 44.4 min | median-model R2 −0.066
```

	quantile	pinball loss (model)	pinball loss (naive quantile)	skill vs naive
0	0.100	5.283	5.296	0.003
1	0.500	21.827	22.177	0.016
2	0.900	20.042	20.814	0.037

```
[saved] 26_duration_intervals.png
```



23. Fairness re-framed

Carrier differences are large in A2. They are **evidence about the model's consistency across operating groups**, not, on their own, evidence that passengers are treated unfairly: carriers differ in schedule structure, hub geography and base delay rates, and carrier is an operational grouping, not a protected attribute of any person. The tables below therefore report, for each group, *base rate*, *calibration* (does a 30% score mean ~30% delayed for this group?), AUC, and the error-rate gaps at one global threshold. Because demographic-parity gaps largely mirror base-rate gaps, the table also shows that relationship.

As a mitigation planned in A2, **per-group thresholds** (equal opportunity: same true-positive rate on validation data) are tested for carriers.

```
test_view = d4.loc[m_test, ["carrier", "origin_hub_tier", "origin_lat", "origin_lon", "arr_del15"]].copy()
test_view["quadrant"] = np.where(test_view["origin_lat"] >= 39, "N", "S") + np.where(test_view["origin_lon"] <=
test_view["p"] = p_te
val_view = d4.loc[m_val, ["carrier"]].copy(); val_view["p"] = p_val; val_view["y"] = y_val
T_GLOBAL = thr_f1


def group_table(col, thr=T_GLOBAL, min_n=200):
    rows = []
    for g, gdf in test_view.groupby(col, observed=True):
        if len(gdf) < min_n:
            continue
        y, p = gdf["arr_del15"].to_numpy(), gdf["p"].to_numpy()
        tp, fp, fn, tn = _confusion(y, p, thr)
        rows.append({"group": str(g), "n": len(gdf), "base_rate": y.mean(), "mean_predicted": p.mean(),
                     "calibration_gap": p.mean() - y.mean(), "ECE": _ece(y, p),
                     "auc": roc_auc_score(y, p), "tpr": tp / max(tp + fn, 1), "fpr": fp / max(fp + tn, 1),
                     "flagged_rate": (tp + fp) / len(y), "precision": tp / max(tp + fp, 1)})
    t = pd.DataFrame(rows).sort_values("n", ascending=False).reset_index(drop=True)
    gaps = {"equalised_odds_gap": max(t["tpr"].max() - t["tpr"].min(), t["fpr"].max() - t["fpr"].min()),
            "demographic_parity_gap": t["flagged_rate"].max() - t["flagged_rate"].min(),
            "base_rate_gap": t["base_rate"].max() - t["base_rate"].min(),
            "corr(base_rate, flagged_rate)": float(np.corrcoef(t["base_rate"], t["flagged_rate"])[0, 1]) if len(
            "max_abs_calibration_gap": t["calibration_gap"].abs().max()}
    return t, gaps
```

```
fair4, fair_gaps = {}, {}
for col, label in [("carrier", "Carrier"), ("origin_hub_tier", "Origin hub tier"), ("quadrant", "Geographic quad
    fair4[label], fair_gaps[label] = group_table(col)
    fair4[label].to_csv(A4_DIR / f"a4_fairness_{col}.csv", index=False)
gap_df = pd.DataFrame(fair_gaps).T
display(gap_df.style.format("{:.3f}"))
gap_df.to_csv(A4_DIR / "a4_fairness_gap_summary.csv")
display(fair4["Carrier"].style.format({c: "{:.3f}" for c in fair4["Carrier"].columns if c not in ("group", "n")}

# Mitigation: per-carrier thresholds that equalise validation TPR with the global operating point
tp, fp, fn, tn = _confusion(y_val, p_val, T_GLOBAL)
target_tpr = tp / max(tp + fn, 1)
thr_by_carrier = {}
for c, g in val_view.groupby("carrier", observed=True):
    if len(g) < 200 or g["y"].sum() < 20:
        continue
    pos = np.sort(g.loc[g["y"] == 1, "p"].to_numpy())
    thr_by_carrier[str(c)] = float(np.quantile(pos, 1 - target_tpr))      # threshold at which this carrier's va
test_view["p_group_thr"] = test_view["carrier"].astype(str).map(thr_by_carrier).fillna(T_GLOBAL)
rows = []
for c, g in test_view.groupby("carrier", observed=True):
    if str(c) not in thr_by_carrier:
        continue
    y = g["arr_del15"].to_numpy(); flag = g["p"].to_numpy() >= g["p_group_thr"].to_numpy()
    tp_ = int((flag & (y == 1)).sum()); fp_ = int((flag & (y == 0)).sum()); fn_ = int((~flag & (y == 1)).sum());
    rows.append({"carrier": str(c), "threshold": thr_by_carrier[str(c)], "tpr": tp_ / max(tp_ + fn_, 1), "fpr":
                 "flagged_rate": flag.mean(), "precision": tp_ / max(tp_ + fp_, 1)})
mit = pd.DataFrame(rows)
mit = mit[mit["carrier"].isin(fair4["Carrier"]["group"])].reset_index(drop=True)
before = fair4["Carrier"].set_index("group").loc[mit["carrier"]]
print(f"Per-carrier thresholds (target validation TPR {target_tpr:.2f}):")
print(f"  TPR gap   : {before['tpr'].max() - before['tpr'].min():.3f} -> {mit['tpr'].max() - mit['tpr'].min():.3
print(f"  FPR gap   : {before['fpr'].max() - before['fpr'].min():.3f} -> {mit['fpr'].max() - mit['fpr'].min():.3
print(f"  flag-rate gap: {before['flagged_rate'].max() - before['flagged_rate'].min():.3f} -> {mit['flagged_rate
display(mit.style.format({c: "{:.3f}" for c in mit.columns if c != "carrier"}))
mit.to_csv(A4_DIR / "a4_fairness_per_carrier_threshold_mitigation.csv", index=False)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
t = fair4["Carrier"].sort_values("base_rate")
axes[0].scatter(t["base_rate"], t["flagged_rate"], s=t["n"] / t["n"].max() * 400, alpha=.7)
for _, r in t.iterrows():
    axes[0].annotate(r["group"], (r["base_rate"], r["flagged_rate"]), fontsize=8)
axes[0].set_xlabel("Observed delay rate (base rate)"); axes[0].set_ylabel(f"Share flagged at threshold {T_GLOBAL
axes[0].set_title("Flagging differences largely follow base-rate differences")
axes[1].barh(t["group"], t["calibration_gap"], color="#55A868")
axes[1].axvline(0, color="black", linewidth=1)
axes[1].set_xlabel("Mean predicted - observed delay rate (calibration gap)")
axes[1].set_title("Per-carrier calibration after calibration step")
save_fig(fig, "27_fairness_base_rate_and_calibration")
```